

红外干涉法测量外延层厚度的建模分析

摘要

本文通过研究红外干涉法测外延层厚度问题,提出了一种基于牛顿迭代反演反射率模型参数、同基于波谷周期关系进行数值计算厚度交叉印证的解决路径。

对于问题一,该方案首先基于碳化硅和硅的先验折射率物理模型和双光束干涉的反射强度公式,设计了一个基于 Drude 折射率模型、给出了直接计算外延层厚度的公式;

对于问题二,在基于折射率和反射强度公式建立的模型上,设计了两个求解路线 (1) 周期波谷直接算法:通过反射率表达式给出的理论震荡周期和厚度 d 的关系建立 d 的显式表达式,在附件 1、2 测得的反射率曲线上通过样条插值取出准确极值点坐标值,计算多组厚度 d 并用统计方法给出最终厚度计算值为 $7.544\mu\text{m}$ (2) 牛顿迭代反演法:以已经建立的反射率表达式 R_d 作为依赖厚度 d 参数曲线,通过用牛顿迭代拟合 R_d 与实验测得的反射率曲线,给出厚度 d 的迭代解为; $7.295\mu\text{m}$ 作为直接计算值 $7.544\mu\text{m}$ 的较好印证值

对于问题三,解答路线是 (A) 先建立多光干涉模型,分析多光束效应显著的物理必要条件,并给出判断多光束效应对原模型求解精度影响大小的评判指标 G , (B) 基于前两问的建模和给出的波谷直接算法和牛顿迭代反演法,代入硅晶片的折射率经验曲线,建立确定硅外延层厚度计算的数学模型和算法。给出附件 3、4 数据的厚度计算值为 $3.453\mu\text{m}$ (直接计算) 和 $3.414\mu\text{m}$ (牛顿迭代),最终以 $3.432\mu\text{m}$ 作为附件 3、4 硅晶片的厚度计算结果 (C) 通过 G 值检验和机器学习检验,用 (B) 中初步计算结果和参与计算的量考察多光束效应对附件 3、4 所测数值影响的强弱 (D) 对附件 1、2 所测数据评估多光束效应,并在波谷直接算法和牛顿迭代反演法上都进行模型修正,以尝试消除多光束效应的影响,得出在修正模型下计算得厚度和第二问中计算得到结果差距极小,因而判断附件 1、2 中多光束干涉效应较弱,最终可信的厚度值是 $7.455\mu\text{m}$

关键词：折射率建模, 牛顿迭代反演, 三次样条拟合, 机器学习

1 问题背景与重述

1.1 问题的背景

碳化硅是一种新型半导体材料, 在某些碳化硅器件的制备过程中, 需要在高掺杂浓度的基底上生长一层低掺杂浓度的外延层。测量碳化硅的外延层厚度对器件的制备至关重要, 我们需要一个精确易得的碳化硅外延层厚度测量方法。

红外干涉法利用等倾干涉的原理: 由于两束反射光之间的相位差受到光波长影响, 红外光入射碳化硅晶圆表面的反射谱呈现出交替相消相长的特性, 也即干涉条纹, 通过该方法可以无损测量碳化硅外延层的厚度。在一定条件下, 光束可能在外延层表面和衬底表面发生多次折射和反射, 进而形成多光束干涉, 从而降低厚度测量的精度。

利用红外干涉法计算外延层厚度需要碳化硅的折射率作为参量, 而碳化硅的折射率受到掺杂浓度和入射光的波长等参数的影响, 能否对碳化硅折射率进行精确建模直接影响到厚度测量精度和可靠性。在固体物理研究中, 通常基于 Drude 模型对不同掺杂浓度碳化硅的复介电常数建立数学模型, 进而得出一定掺杂浓度下碳化硅的折射率-波数曲线。

1.2 问题的重述

问题 1: (1) 仅考虑光在外延层和衬底之间的单次反射、透射, 基于红外光的反射率光谱情形, 建立确定外延层厚度的数学模型。

问题 2: (1) 使用问题 1 建立的模型, 设计算法, 用红外光反射率光谱计算碳化硅外延层的厚度, (2) 对附件提供的不同入射角的碳化硅片的光谱实测数据进行分析计算, 给出计算结果 (3) 评估结果的可靠性。

问题 3: (1) 实际情况中, 光波可以在外延层和空气界面以及外延层和衬底界面多次反射和透射, 推导产生多光束干涉的必要条件, (2) 分析多光束干涉是否会影响外延层厚度的计算精度。 (3) 根据必要条件分析附件 3 和附件 4 的硅晶圆片数据是否体现出多光束干涉, 建立数学模型和算法, 计算硅外延层厚度。 (4) 分析碳化硅晶圆片中是否存在多光束干涉, 如果存在, 则消除多光束干涉的影响, 并给出新的计算结果。

2 问题分析

2.1 问题一: 双光束干涉模型与厚度计算公式

本题首先需要建立红外干涉法下的物理模型。在假设仅有空气/外延层与外延层/衬底两界面的单次反射时, 可用双光束干涉模型描述反射率。其关键是基于菲涅尔公式写出反射振幅, 结合外延层厚度 d 与相位延迟 δ 的关系, 得到反射率 $R(\tilde{\nu}, d)$ 的解析式。这样, 膜厚 d 就成为反射率曲线的决定性参数。问题一的核心是将折射率模型 $n(\tilde{\nu}, N)$ 与干涉公式结合, 推导出外延层厚度的计算表达式。

2.2 问题二：两种厚度反演方法与可靠性分析

在问题一模型基础上，需要从实际反射率光谱数据反推出膜厚。为此设计了两条路线：

1. **周期波谷直接计算法**：利用干涉极值点之间的相位差恒为 2π 的特征，推导出厚度 d 与相邻波谷波数间隔的显式关系。通过样条插值获取精确极值点，再逐对计算出多个 d ，最后用统计方法得出平均厚度及置信区间。
2. **牛顿迭代反演法**：将反射率表达式 $R(\bar{\nu}, d, N_1, N_2)$ 与实测曲线 $R_{\text{exp}}(\bar{\nu})$ 拟合，构造残差平方和作为目标函数，以 d, N_1, N_2 为变量进行迭代优化。通过网格搜索设定初值，避免局部极值陷阱。最终得到稳定的迭代解，输出厚度 d 。

两种方法相互验证，并结合统计指标（标准差、置信区间、变异系数）来评估厚度求解结果的可靠性。

2.3 问题三：多光束干涉效应与修正

实际情况中，光在外延层内会发生多次往返反射，形成多光束干涉。相比双光束模型，多光束干涉在反射率表达式中引入分母几何级数项。问题三的思路是：

1. 推导多光束干涉的等效反射率公式 $R_{\text{eff}}(\bar{\nu})$ ，并定义表征效应显著性的参数 G 。
2. 通过比较多光束与双光束模型的差异，判断多光束效应对厚度计算精度的影响，特别是在外延层折射率较大、吸收较弱时， G 值增大，效应显著。
3. 对附件数据，先用双光束模型计算厚度，再引入修正公式

$$R_{2b}(\delta) = R_{\text{eff}}(\delta) |1 + r_{01}r_{12}e^{i\delta}|^2$$

还原得到双光束等效反射率，重新进行厚度反演，从而减弱多光束效应带来的系统偏差。

这样既能解释多光束效应的物理条件，也能通过修正得到更接近真实的厚度值。

3 模型假设

模型假设主要包括

3.1 碳化硅和硅固体物理特性假设

(1) 掺杂浓度假设：题目所指向的现实背景中，碳化硅和硅在衬底和外延层上都有一定的掺杂浓度，而掺杂浓度对折射率造成影响。为尽可能简化模型，我们假设衬底和外延层的掺杂浓度分别为常数，且在衬底和外延的交界面上突变

(2) 极性假设：晶体的光学声子与光波的耦合现象受到晶体极性的显著影响，为了简化模型的同时贴合实际情况，做如下假设：硅作为非极性材料，即使在高掺杂浓度下我们仍假设其 LO/TO 光学声子兼并，在折射率模型建立中也不考虑相关效应；碳化硅作为极性较强的材料，我们将在数学模型中考虑红外光与光学声子的耦合效应。

3.2 入射光偏振情况假设

反射光束干涉模型的建立依赖于介质表面的 Fresnel 公式，而不同偏振光在斜入射情况下得到的结果有所不同，为得到具有一般意义的答案，并尽可能简化求解过程，我们假设入射光为完全偏振光，即 s 光和 p 光的光照强度各占 50%。

4 本文符号约定

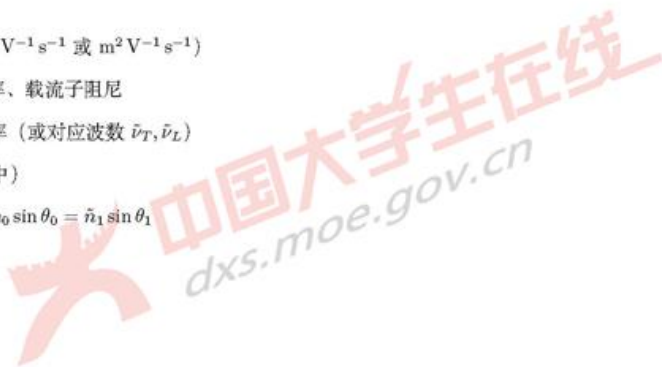
符号表

拉丁字母

λ	真空波长，单位：cm
σ_m	以 m^{-1} 计的波数， $\sigma_m = 100 \tilde{\nu}$
$\tilde{n}_1 = n_1 + i\kappa_1$	膜层复折射率（实部 n_1 ，消光系数 κ_1 ）
$\tilde{\nu}$	波数（常用 cm^{-1} ），与 λ 关系： $\tilde{\nu} = 1/\lambda$
d	薄膜厚度，单位： μm
E_r/E_0	反射/入射电场的复振幅比
n_0	入射介质折射率（通常为空气 = 1）
n_2	基底折射率（可复数）
r_{exact}	多光束（Airy）等效反射振幅
r_{2-beam}	两光束（单次往返）等效反射振幅
ν	频率，单位：Hz

希腊字母

β	单程相位， $\beta = \frac{2\pi}{\lambda} \tilde{n}_1 d \cos \theta_1$
δ	往返相位， $\delta = 2\beta$
γ_{ph}	声子阻尼
μ	载流子迁移率（ $cm^2 V^{-1} s^{-1}$ 或 $m^2 V^{-1} s^{-1}$ ）
ω_p, γ_e	Drude 等离子角频率、载流子阻尼
ω_T, ω_L	TO/LO 声子角频率（或对应波数 $\tilde{\nu}_T, \tilde{\nu}_L$ ）
$\phi = \theta_0$	入射角（在介质 0 中）
θ_1	膜内折射角，满足 $n_0 \sin \theta_0 = \tilde{n}_1 \sin \theta_1$



ε_{∞} 高频介电常数

N 掺杂/自由载流子浓度 (cm^{-3})

N_1 外延层掺杂浓度

N_2 衬底掺杂浓度

运算符与常量

i 虚数单位, $i^2 = -1$ (与条纹阶差 $i \in \mathbb{Z}$ 区分)

π 圆周率

c 真空光速, $c = 299792458 \text{ m/s}$

下标与上标

$(s), (p)$ 偏振标记: s 偏振 / p 偏振

0, 1, 2 介质编号: 入射介质/薄膜/基底

$\Re\{\cdot\}, \Im\{\cdot\}$ 复数的实部/虚部 (或 $\kappa = \Im \tilde{n}$)

5 模型建立与求解

5.1 问题 1 模型建立:

5.1.1 折射率模型建立

红外干涉法测量碳化硅外延层厚度依赖于碳化硅的折射率作为参量, 而碳化硅材料的折射率同时受到载流子掺杂浓度、入射光波数等因素的影响, 为了精确计算题目情景下的碳化硅折射率曲线, 我们查阅资料, 借助固体物理的先验知识建立了折射率 n 关于载流子掺杂浓度 N 和入射波波数 $\tilde{\nu}$ 的数学模型, 即:

$$n = n(\tilde{\nu}, N) \quad (1)$$

光束的折射率由材料的复介电常数确定:

$$n = \sqrt{\Re(\varepsilon(\tilde{\nu}))} \quad (2)$$

在固体物理中, 通常应用 Drude 经典模型研究载流子对材料电磁特性的影响, 进而求出复介电常数 $\varepsilon_{\text{Drude}}$. 与此同时, 考虑到碳化硅晶体具有较强极性, 在题目所求的中远红外波段 ($2.5\mu\text{m} \sim 25\mu\text{m}$) 下, 碳化硅晶体光学声子与红外光耦合而产生的强吸收峰不可忽略, 因此加入以 LO/TO 光学声子频率为参量的修正项 $\varepsilon_{\text{corrected}}$ 对复介电常数计算进行修正. 从而给出复介电常数的计算公式:

$$\varepsilon(\tilde{\nu}) = \varepsilon_{\text{Drude}} + \varepsilon_{\text{corrected}} = \varepsilon_{\infty} \left(1 + \frac{\tilde{\nu}_L^2 - \tilde{\nu}_T^2}{\tilde{\nu}_T^2 - \tilde{\nu}^2 - i\Gamma\tilde{\nu}} - \frac{\tilde{\nu}_P^2}{\tilde{\nu}(\tilde{\nu} - i\gamma)} \right) \quad (3)$$

上述公式中, Drude 等离子频率 $\tilde{\nu}_p$ 和载流子阻尼 γ_e 由常见公式计算得到:

$$\tilde{\nu}_p = \sqrt{\frac{Nq^2}{m^* \varepsilon_0 \varepsilon_\infty}} \quad (4)$$

$$\gamma = \frac{q}{m^* \mu(N)} \quad (5)$$

对于载流子迁移率 μ , 由于未知载流子掺杂浓度范围, 我们利用普遍适用的 CT 经验公式 (Caughey-Thomas 模型) 进行计算:

$$\mu = \mu_{\min} + \frac{\mu_{\max} - \mu_{\min}}{1 + \left(\frac{N}{C_r}\right)^\alpha} \quad (6)$$

联立上述各式, 我们即得到了碳化硅折射率 n 关于光束波数 $\tilde{\nu}$ 及载流子掺杂浓度 N 的数学模型 (这里的折射率依然是复数, 用于在波动光学中进行计算):

$$n(\tilde{\nu}, N) = \sqrt{\varepsilon_\infty \left(1 + \frac{\tilde{\nu}_L^2 - \tilde{\nu}_T^2}{\tilde{\nu}_T^2 - \tilde{\nu}^2 - i\Gamma\tilde{\nu}} - \frac{Nq^2}{m^* \varepsilon_0 \varepsilon_\infty} \right)} \quad (7)$$

建立模型所用的物理常数和材料参量取值均由常见光学手册和权威论文给出, 参见支撑材料文件“物理常数和材料参量”部分。

根据上述建立的关于碳化硅材料的折射率数学模型, 我们取一些给定的掺杂浓度绘制出了折射率 n 随入射光波数的变化曲线, 从曲线中可以看到, 在题目情景的红外光波段下, 碳化硅折射率数值符合常见的经典值, 且在波数 $\tilde{\nu} = 793 \text{ cm}^{-1}$ 处出现陡增的尖峰, 表现出符合光学声子对红外光吸收峰的性质。

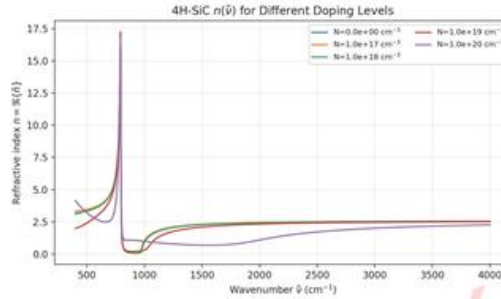


图 1: 碳化硅材料在给定掺杂浓度下折射率 (实部) 随入射光波数的变化曲线

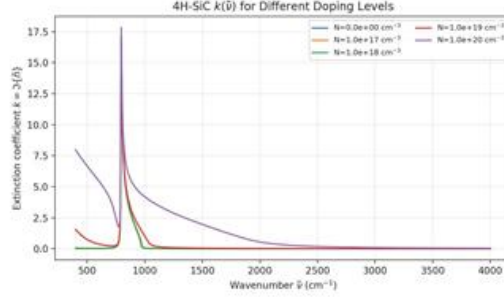


图 2: 碳化硅材料在给定掺杂浓度下折射率（虚部）随入射光波数的变化曲线

5.1.2 反射光束干涉模型

在波动光学中, 根据 Fresnel 公式, 在界面 $i|j$ 上发生折射与反射时, 复振幅系数为 ($c_i = \cos \theta_i$):

$$r_{ij}^{(s)} = \frac{\tilde{n}_i c_i - \tilde{n}_j c_j}{\tilde{n}_i c_i + \tilde{n}_j c_j}, \quad t_{ij}^{(s)} = \frac{2\tilde{n}_i c_i}{\tilde{n}_i c_i + \tilde{n}_j c_j}, \quad (8)$$

$$r_{ij}^{(p)} = \frac{\tilde{n}_j c_i - \tilde{n}_i c_j}{\tilde{n}_j c_i + \tilde{n}_i c_j}, \quad t_{ij}^{(p)} = \frac{2\tilde{n}_i c_i}{\tilde{n}_j c_i + \tilde{n}_i c_j}. \quad (9)$$

设薄膜折射率为 $\tilde{n}_1$, 厚度为 d , 入射角为 ϕ , 膜内折射角 θ_1 由 Snell 定律

$$n_0 \sin \phi = \tilde{n}_1 \sin \theta_1$$

给出。通过小量近似法给出, 光在薄膜中一次往返的相位延迟为

$$\delta(\lambda) = \frac{4\pi d}{\lambda} \tilde{n}_1 \cos \theta_1 = \frac{4\pi d}{\lambda} \sqrt{\tilde{n}_1^2 - n_0^2 \sin^2 \phi}. \quad (10)$$

在仅考虑光束在外延层和衬底发生一次透射、反射的情况下, 实际发生干涉的光束为双光束, 也即:

1. 从顶界面 $0|1$ 直接反射的光, 振幅表示为 $r_{01}^{(\text{pol})}$;
2. 透射进入薄膜, 在界面 $1|2$ 反射后再次透射出的光, 振幅表示为 $t_{01}^{(\text{pol})} t_{10}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}$ 。

因此, 总反射光场振幅为

$$r_{2\text{-beam}}^{(\text{pol})}(\delta) = r_{01}^{(\text{pol})} + t_{01}^{(\text{pol})} t_{10}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}, \quad (11)$$

联立

$$r_{2\text{-beam}}^{(\text{pol})}(\delta) = r_{01}^{(\text{pol})} + t_{01}^{(\text{pol})} t_{10}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}, \quad (12)$$

根据模型假设, 考虑入射光为完全偏振光, 因此最终反射率表现为 s 波反射率和 p 波反射率的平均, 对应公式为

$$R^{(\text{pol})}(\delta) = |r_{2\text{-beam}}^{(\text{pol})}(\delta)|^2, \quad R_{\text{avg}} = \frac{1}{2}(R^{(s)} + R^{(p)}). \quad (13)$$

综合上述各式,并结合 5.1.1 中的折射率模型,我们建立了双光束干涉的近似反射模型, $R_{\text{avg}}(\bar{\nu})$ 表现为以外延层厚度 d 、外延层掺杂浓度 N_1 、衬底掺杂浓度 N_2 为参数的参数曲线,可用于薄膜厚度的反演计算。

5.1.3 外延层厚度确定方法 (周期波谷直接计算法)

由反射谱强度公式知在弱吸收情况下,反射谱谷(或峰)之间相邻极值的相位差恒为 2π 。由式 (10) 可知相位延迟为

$$\delta(\bar{\nu}) = 4\pi d (100 \bar{\nu}) \sqrt{n^2(\bar{\nu}) - n_0^2 \sin^2 \phi},$$

其中 $\bar{\nu}$ 为波数 (单位 cm^{-1}), $n(\bar{\nu})$ 为折射率, ϕ 为入射角。

若在两个反射谱谷(或峰)处波数分别为 $\bar{\nu}_i, \bar{\nu}_j$, 且对应干涉级次差为整数 $i = m_j - m_i$, 则有

$$\delta(\bar{\nu}_j) - \delta(\bar{\nu}_i) = 2\pi i.$$

由此解得薄膜厚度:

$$d = \frac{i}{2 \left[\bar{\nu}_j \sqrt{n^2(\bar{\nu}_j) - n_0^2 \sin^2 \phi} - \bar{\nu}_i \sqrt{n^2(\bar{\nu}_i) - n_0^2 \sin^2 \phi} \right]} \quad (14)$$

该公式给出了在已知折射率曲线与入射角的条件下,通过反射频谱极值位置直接计算膜厚 d 的方法。

5.1.4 模型总结

综上,问题 1 通过对碳化硅折射率的物理建模与双光束干涉模型的推导,建立了红外反射率与外延层厚度之间的数学联系。首先,利用包含 Drude 自由载流子项与 LO/TO 光学声子修正项的介电函数模型,得到折射率函数 $n(\bar{\nu}, N)$, 保证了模型在中远红外波段下的物理合理性。其次,基于菲涅尔公式与相位延迟关系,推导出双光束干涉的反射率表达式 $R_{\text{avg}}(\bar{\nu}; d, N_1, N_2, \phi)$, 明确了反射率曲线由厚度、掺杂浓度和入射角等物理参量共同决定。最后,通过分析相邻极值点间的相位差关系,给出了外延层厚度 d 的直接计算公式,为后续的数值求解与实验数据拟合提供了基础。该模型兼具物理解释力与计算可操作性,是后续问题 2 和问题 3 的出发点。

5.2 问题 2 模型建立与求解

在问题 1 给定的模型上求解具体的 d 值,我们给出两条可行的路径:

(1) 周期波谷直接计算法该方法直接用 d 的显示表达式,通过在反射谱上取出多组相邻的周期波谷极值点,代入数据直接计算得相应的厚度值,再对计算结果进行统计分析,结合公式误差来源(其证明在“模型检验与合理性分析”中)舍弃计算误差较大的结果,接着给出直接计算法的误差限、结果可信水平、置信区间

(2) 牛顿迭代反演模型法该方法观点下,我们在给定掺杂浓度 N_1, N_2 后将唯一确定衬底和外延层的折射率函数 $n_i(\bar{\nu})$, 进而对于每一个厚度 d , 我们能计算出唯一的反射率曲线 $R(d, n_1(\bar{\nu}), n_2(\bar{\nu}))$,

对于一个接近真实值的厚度 d_q , 其对应的反射率曲线 $R(d_q, n_1(\bar{\nu}), n_2(\bar{\nu}))$ 会非常接近于观测到的反射率曲线 R_0

我们以 $\frac{1}{2} \sum_{i=1}^M (R_{calc}(\bar{\nu}_i, p) - R_{exp}(\bar{\nu}_i))^2$ 作为需要极小化的损失函数 F 其中, M 是真实测得的参数点的数量, 因外延层和衬底折射率是掺杂浓度 N_1, N_2 为参量的曲线, 给定 N_1, N_2 和 d 后 $R_{exp}(\sigma)$ 被完全确定。我们以这三个量为训练参数最优化损失函数 F 。

当 F 极小的时候, 理论计算的反射率曲线最贴近于附件 1, 2 给出的真实反射率曲线。此时输出的参数 N_1, N_2 是此块碳化硅材料外延层和衬底的掺杂浓度一个好的近似, d 是接近于真值厚度的一个值

方法 1: 周期波谷直接计算法

5.2.1 数据预处理

通过三次样条插值平滑化函数并通过对样条函数求数值导数找到其局部最小值点, 样条函数的引入是为了: (1) 重建反射率函数滤去毛刺高频噪声, 便于识别曲线大势趋上的周期波动, (2) 提供一个确定的平滑函数表达式, 便于通过求导找到更准确波谷频数的参考值:

如图所示, 被标出的点是样条函数上取出表征反射率周期波谷的点 (绿色是后续经计算得到可信厚度 d 值的点, 红色是计算得到偏差较大的厚度 d , 且经过误差限分析 (在模型检验及合理性分析部分给出证明) 认为和前后波谷点用于计算会有较大误差的点)。

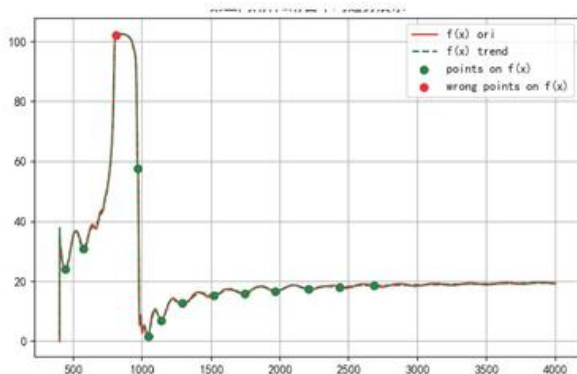


图 3: 附件 2 滑动窗口样条函数和极值点-全局情况

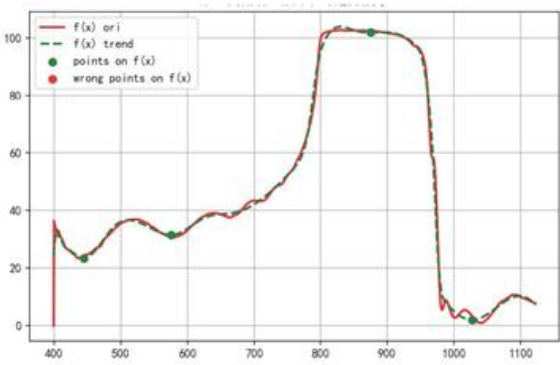


图 4: 附件 2 滑动窗口样条和极值点-局部情况

5.2.2 厚度计算及结果

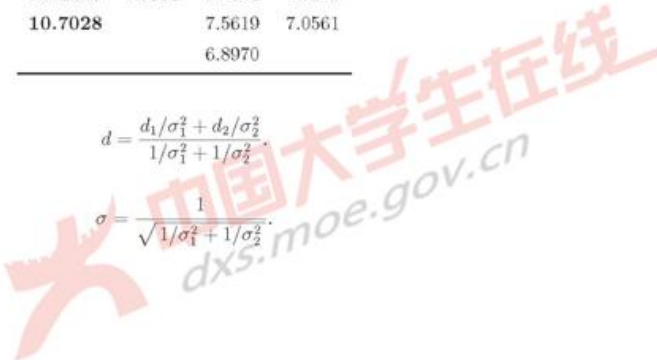
用滑动窗口样条拟合碳化硅的反射率随波数的变化便于取出极值点，对所有取出的波谷按照顺序，用相邻两波谷频数来计算外延层厚度，得到多个外延层厚度，利用 3σ 原则剔除差距较大可疑值之后对其进行统计分析。最终用直接计算法的厚度将是通过统计方法处理以上数据的结果：

表 1: 计算得到的厚度 d （标准偏差大于 3σ 的异常值标记黑体）

厚度 d 值 (μm)			
入射角 10°		入射角 15°	
7.6298	7.6100	7.4837	7.5434
7.791	7.370	7.4360	7.4237
3.770	7.3104	7.9480	7.4515
7.7617	7.4367	1.1918	7.4464
7.6774	7.2732	1.4167	7.3153
7.8101	7.8501	7.8162	7.3767
20.5125	7.6394	7.7824	7.7172
10.7028		7.5619	7.0561
		6.8970	

$$d = \frac{d_1/\sigma_1^2 + d_2/\sigma_2^2}{1/\sigma_1^2 + 1/\sigma_2^2}$$

$$\sigma = \frac{1}{\sqrt{1/\sigma_1^2 + 1/\sigma_2^2}}$$



得

$$d = 7.544, \sigma = 0.170$$

5.2.3 可靠性分析

对厚度组进行了统计分析, 计算了平均值、标准差、不确定度、置信区间和变异系数等量。

表 2: 剔除异常值之后修正模型碳化硅外延层厚度值的统计值

统计量	入射角 10°	入射角 15°
数据点数 L	11	15
自由度 n	10	14
平均厚度	7.575856	7.48711
标准偏差 σ_d	0.212318	0.284504
标准误差 $\sigma_{\bar{d}}$	0.067141	0.076037
$t_{\text{crit}}(95\%)$	2.262157	2.160369
CI _{95%} (low)	7.423973	7.322842
CI _{95%} (high)	7.727739	7.651377
离散系数 CV(%)	2.802562	3.799922

均值计算 剔除可疑点并认为剩余样本近似独立同分布, 记样本数为 L ,

$$\bar{d} = \frac{1}{L} \sum_{i=1}^L d_i, \quad s_d = \sqrt{\frac{1}{\nu} \sum_{i=1}^L (d_i - \bar{d})^2},$$

其中自由度 ν 一般取 $\nu = L - 1$

不确定度计算

标准偏差:

$$\sigma_d = \sqrt{\frac{1}{L-2} \sum_{i=1}^{L-1} (d_i - \bar{d})^2} \quad (33)$$

标准误差:

$$\sigma_{\bar{d}} = \frac{\sigma_d}{\sqrt{L-1}} \quad (34)$$

置信区间计算 在未知方差的小样本条件下,

$$T = \frac{\bar{d} - \mu_d}{s_d / \sqrt{L}} \sim t_{\nu},$$

从而真实平均厚度 μ_d 的 $100(1 - \alpha)\%$ 置信区间为

$$\text{CI}_{1-\alpha}: \mu_d \in \bar{d} \pm t_{1-\alpha/2, \nu} \frac{s_d}{\sqrt{L}} \quad (15)$$

当 $L \geq 30$ 或正态性检验（如 Shapiro-Wilk）通过时， t 分布可近似为标准正态，式 (15) 中的分位数可换用 $z_{1-\alpha/2}$ 。

变异系数检查为表征相对离散程度，使用变异系数

$$CV = \frac{s_d}{\bar{d}} \times 100\%.$$

经验阈值： $CV < 3\%$ 为优秀重复性， $3\% \sim 5\%$ 为良好， $5\% \sim 10\%$ 为基本可接受；若 $CV > 10\%$ 则提示离散性较高，宜复测或检查模型与数据处理（例如在折射率色散陡峭区 $|dn/d\nu|$ 较大时，厚度解对频率/波数扰动更敏感）。

根据表 2 的计算结果，利用两个入射角下得到的平均厚度和标准偏差，用逆方差公式计算得最终结果

$$d = 7.455, \sigma = 0.170$$

方法 2: 牛顿迭代反演参数法

5.2.4 迭代反演参数方法介绍

在材料科学与工程领域，用观测到的实验数据反演材料的物理参数是重要的任务。通常需要构建非线性物理模型，使用最优化算法拟合实验数据。

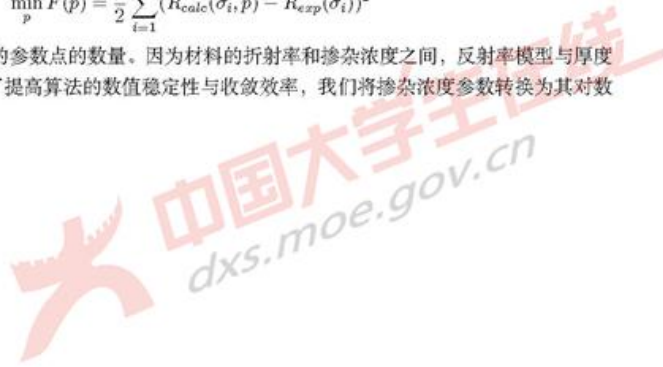
而本研究需要利用红外反射光谱的数据，确定薄膜厚度及各层掺杂的浓度。我们采用网格搜索和牛顿迭代结合的搜索优化算法，实现非线性空间内的最优参数搜索。

5.2.5 问题构建

我们的目标是找到一组参数 $p = [d, N_1, N_2]$ （分别代表外延层厚度、外延层掺杂浓度和衬底掺杂浓度），通过同时优化掺杂浓度（进而调整折射率随频率变化的参数曲线形状）和参考的外延层厚度使得理论计算的反射率 $R_{calc}(\sigma, p)$ 与实验测量的反射率 $R_{real}(\sigma)$ 之间的残差平方和最小，此即，当在迭代输出的厚度和掺杂浓度下，若理论计算反射曲线接近于实验测量曲线，则认为迭代得到的厚度较好的反映了真实的厚度值。这是一个非线性最小二乘优化问题：

$$\min_p F(p) = \frac{1}{2} \sum_{i=1}^M (R_{calc}(\sigma_i, p) - R_{exp}(\sigma_i))^2$$

其中， M 是真实测得的参数点的数量。因为材料的折射率和掺杂浓度之间，反射率模型与厚度之间存在非线性关系，为了提高算法的数值稳定性与收敛效率，我们将掺杂浓度参数转换为其对数形式 $\log_{10}(N)$ 进行优化。



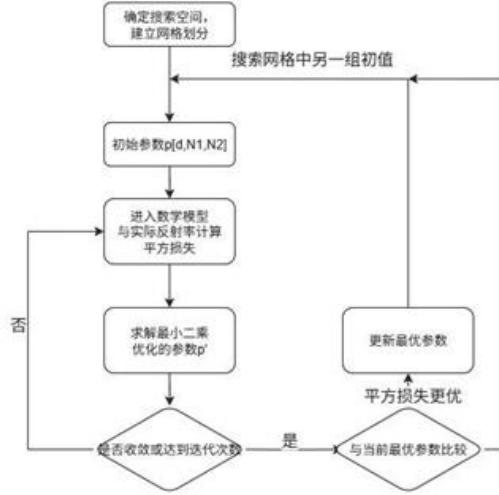


图 5: 结合高斯牛顿优化的网格搜索算法流程

5.2.6 搜索过程

为了克服非线性最小二乘容易陷入局部最优的问题, 本研究采用了网格搜索来设定参数的初值。我们为薄膜厚度 d 、薄膜掺杂浓度 $\log_{10}(N_1)$ 和衬底掺杂浓度 $\log_{10}(N_2)$ 各自设定了搜索范围, 在参数空间表现为网格格点。初始参数从这些格点出发, 进行高斯-牛顿法优化, 可以防止算法停留在某个初值对应的局部最优点。

5.2.7 迭代优化过程和结果

在网络搜索给出参数初值之后, 我们采用高斯-牛顿法对参数进行精细优化。高斯-牛顿法是一种用于解决非线性最小二乘问题的迭代算法, 其优势在于利用一阶导数信息近似二阶导数信息, 从而避免了计算和存储复杂的二阶导数 Hessian 矩阵。

高斯-牛顿法的迭代更新公式可表示为:

$$J(p_k)\Delta p = -f(p_k)$$

其中, p_k 是当前迭代的参数向量, $f(p_k)$ 是残差向量 ($R_{calc}(\sigma, p_k) - R_{exp}(\sigma)$), $J(p_k)$ 是残差向量 $f(p_k)$ 对参数 p_k 的雅可比矩阵 (Jacobian Matrix)。其具体实现步骤如下:

1. 残差计算: 在每次迭代中, 首先根据当前参数 p_k 计算理论反射率, 并得到残差向量 $f(p_k)$ 。目标函数的损失值即为残差向量的平方和。

2. 雅可比矩阵计算: 我们采用有限差分法来近似计算雅可比矩阵 $J(p_k)$ 中的元素。对于每个参数 p_j , 其对应的雅可比列向量通过微小扰动参数 p_j 来计算:

$$J_{i,j} = \frac{\partial f_i(p)}{\partial p_j} \approx \frac{f_i(p + h \cdot e_j) - f_i(p)}{h}$$

其中 e_j 是第 j 个分量为 1 的单位向量, h 是一个小的扰动步长。

3. 参数更新: 获得雅可比矩阵 $J(p_k)$ 和残差向量 $f(p_k)$ 后, 求解线性最小二乘问题 $J(p_k)\Delta p = -f(p_k)$, 得到参数更新量 Δp 。

4. 收敛性判断: 算法通过检查连续两次迭代之间参数更新量 $\|p_{k+1} - p_k\|$ 的范数是否小于预设的容许误差来判断是否达到收敛。

反演得到 4H-SiC 薄膜的厚度以及薄膜和衬底的掺杂浓度。

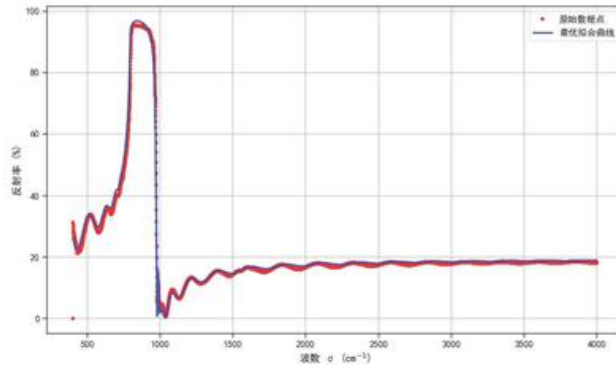


图 6: 反演参数曲线与真实碳化硅 10 度入射角反射率曲线的匹配情况

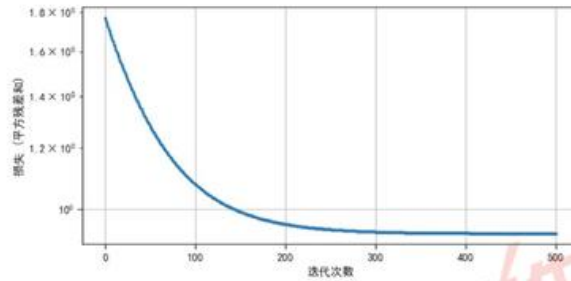


图 7: 高斯-牛顿迭代误差收敛情况

从图 6 中可以看到, 根据模型建立的参数曲线与真实实验数据吻合度极高, 反映了我们建立的物理模型和使用的参数的有效性, 即碳化硅折射率和双光束干涉模型较好解释了真实反射率曲线的

产生原因, 并通过拟合真实反射率曲线学习到了一个可以较好表征真实外延层厚度的值 d 。拟合得到的参数:

表 3: 反演得到的参数 (碳化硅入射角 10°)

反演参数	反演值
外延层厚度 $d(\mu\text{m})$	7.295
外延层掺杂浓度 $N_1(\text{cm}^{-3})$	3.41×10^{17}
衬底掺杂浓度 $N_2(\text{cm}^{-3})$	2.95×10^{18}

反演得到外延层厚度 $d = 7.295\mu\text{m}$, 与方法 1 的结果接近, 两者相互印证。

5.2.8 问题 2 模型与求解方法总结

综上所述, 问题 2 的求解主要建立在问题 1 所提出的反射率物理模型之上。我们提出了两条互为印证的计算路径:

- **周期波谷直接计算法:** 基于反射率极值点的周期性规律, 利用相邻波谷频数差与膜厚之间的解析关系, 直接计算得到外延层厚度。通过样条插值与极值点提取实现对实验噪声的抑制, 再结合统计方法剔除异常值, 从而保证结果的稳定性与可信度。
- **牛顿迭代反演法:** 将外延层厚度与掺杂浓度作为待定参数, 构建非线性最小二乘优化问题, 通过网格搜索与高斯-牛顿迭代相结合实现参数的收敛拟合。该方法能够充分利用整个反射率曲线的信息, 具有较强的鲁棒性与全局性。

两种方法各具优势: 直接计算法公式简洁、直观透明, 适合快速获得近似厚度值; 迭代反演法则能够在全局范围内优化参数, 获得与实验数据高度吻合的结果。二者在数值结果上的接近性也从侧面验证了所建物理模型与算法的可靠性, 为进一步分析多光束效应奠定了坚实基础。

5.3 问题 3 模型建立与求解

5.3.1 多光束干涉模型建立

双光束模型忽略了在外延层内多次反射后透射到空气中的光束分量。在多光束干涉模型中, 同样 Drude 模型给出的折射率参数曲线, 但修改反射强度 R 依赖于折射率 n_1, n_2 和厚度 d 的表达式如下:

把在膜内往返的高阶项按几何级数求和, 得到¹

$$\begin{aligned}
 r_{\text{eff}}^{(\text{pol})}(\delta) &= r_{01}^{(\text{pol})} + t_{01}^{(\text{pol})} t_{10}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta} \left(1 + r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta} + \dots \right) \\
 &= \frac{r_{01}^{(\text{pol})} + r_{12}^{(\text{pol})} e^{i\delta}}{1 + r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}}
 \end{aligned} \quad (16)$$

¹ 式中每一轮回都包含一次 1/2 反射与一次 0/1 反射; 若考虑吸收, 可令 $e^{i\delta} \rightarrow e^{i\delta - \frac{4\pi}{\lambda} n_2 d \cos \theta_2}$ 。

其中界面 ij 的非涅尔振幅系数 (s/p) 为

$$r_{01}^{(s)} = \frac{\tilde{n}_0 c_0 - \tilde{n}_1 c_1}{\tilde{n}_0 c_0 + \tilde{n}_1 c_1}, \quad r_{01}^{(p)} = \frac{\tilde{n}_1 c_0 - \tilde{n}_0 c_1}{\tilde{n}_1 c_0 + \tilde{n}_0 c_1}, \quad c_i \equiv \cos \theta_i. \quad (17)$$

反射率曲线

$$R_{\text{eff}}^{(\text{pol})}(\delta) = \left| r_{\text{eff}}^{(\text{pol})}(\delta) \right|^2 \quad (18)$$

5.3.2 多光束干涉效应的强弱讨论

真实的物理模型更贴近于多光束干涉，而非双光束干涉。本节中通过比对多光束干涉和双光束干涉的振幅和反射率曲线，探究什么情况下多光束干涉和双光束干涉模型差异较大，进而多光束干涉的影响不可忽略。 $r_{\text{eff}}^{(\text{pol})}(\delta)$ 对比双光束干涉的反射振幅表达式

$$r_{2\text{-beam}}^{(\text{pol})}(\delta) = r_{01}^{(\text{pol})} + t_{01}^{(\text{pol})} t_{10}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta} e^{-\frac{4\pi}{\lambda} \kappa_1 d \cos \theta_1} \quad (19)$$

由此可知两光束 (19) 与多光束 (16) 的差别。在较小入射角下，二者分子项基本相同，多光束干涉的分母项多了一项复数，影响了反射率曲线的形状。

即

$$\frac{r_{2\text{-beam}}^{(\text{pol})}(\delta)}{r_{\text{eff}}^{(\text{pol})}(\delta)} \approx \frac{1}{1 + r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}} \quad (20)$$

多光束效应显著性表征量的引入 描述反射率曲线形状改变可以直接用分母上增加的复数量的模 G 来考量，其中

$$G \equiv \left| r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} \right| \exp\left(-\frac{4\pi}{\lambda} \kappa_1 d \cos \theta_1\right). \quad (21)$$

用 G 表示的 $r_{\text{eff}}^{(\text{pol})}(\delta)$ 为

$$r_{\text{eff}}^{(\text{pol})}(\delta) = \frac{r_{01}^{(\text{pol})} + r_{12}^{(\text{pol})} e^{i\delta}}{1 + G e^{i(\delta+\phi)}}, \quad \phi \equiv \arg(r_{01}^{(\text{pol})} r_{12}^{(\text{pol})}).$$

$$\frac{R_{\text{eff}}^{(\text{pol})}(\delta)}{R_{2\text{-beam}}^{(\text{pol})}(\delta)} = \left(\left| \frac{r_{\text{eff}}^{(\text{pol})}(\delta)}{r_{\text{eff}}^{(\text{pol})}(\delta)} \right| \right)^2 \approx \left| \frac{1}{1 + G e^{i(\delta+\phi)}} \right|^2$$

式中 ϕ 只是影响相位，不对反射率大小和其随频率振动的周期（即反射率曲线的振动有贡献），而较大的 G 将显著影响反射率，进而使得多光束和双光束模型产生较大差异。下面考虑较大 G 值的产生

G 表达式中的 $\exp\left(-\frac{4\pi}{\lambda} \kappa_1 d \cos \theta_1\right)$ 的物理意义对应于在介质中因透射造成的强度损失由于薄膜内单程相位为

$$\beta = \frac{2\pi}{\lambda} \tilde{n}_1 d \cos \theta_1 = \frac{\omega}{c} \tilde{n}_1 d \cos \theta_1, \quad \delta \equiv 2\beta. \quad (22)$$

若写出吸收（每一次往返的振幅衰减），则

$$e^{i\delta} \rightarrow \underbrace{\exp\left(i \frac{4\pi}{\lambda} n_1 d \cos \theta_1\right)}_{\text{相位}} \underbrace{\exp\left(-\frac{4\pi}{\lambda} \kappa_1 d \cos \theta_1\right)}_{\text{振幅衰减}}. \quad (23)$$

不难知在透射系数 κ_1 (即折射率 n_1 的虚部) 较小时多光束模型和双光束差异显著，多光束干涉的效应愈发明显

对于 G 表达式中的剩余项 r_{01}, r_{12} ，可由菲涅尔系数读出影响 G 的关键因素与结论：

多光束干涉产生必要条件：

- 顶层面（空气 $\rightarrow$ 外延层）反射强：近法线时 $R_{01} \approx \left(\frac{n_1 - n_0}{n_1 + n_0}\right)^2$ 。因此 n_1 越大（与空气差越大）， $|r_{01}|$ 越大，更容易 G 大、出现显著多光束效应。
- 外延层内吸收弱吸收系数 κ_1 越小或远离强吸收带（如 Reststrahlen 带），式 (23) 的衰减越弱，高阶往返保留，多光束的更显著；同时需光源相干长度 $L_c \gtrsim 2n_1 d \cos \theta_1$ 。
- 指定范围的人射角度（较小的人射角）：s 偏振 $R_{01}^{(s)}$ 随入射角 ϕ 增大而增大，因而对于 s 光，较大的人射角造成多光束效应不可忽略；p 偏振在布儒斯特角 $\tan \phi_B = n_1/n_0$ 附近 $R_{01}^{(p)} \rightarrow 0$ ，在远离布儒斯特角的位置，多光束效应较强，不能用双光束模型简单替代。
- 极端角度 / 全反射：若从膜向空气看满足 $\theta_1 > \theta_c = \arcsin(n_0/n_1)$ ，则 $|r_{10}| \approx 1$ ，多光束极强。

从式 (16) 与判据 (21) 可见：当外延层折射率较大（使 R_{01} 高）、吸收小、相干性足、入射角不太大时，反射谱 $R(\lambda)$ 的峰谷形状、深度与两光束模型 (19) 相比会出现显著差异，此时应采用多光束 (Airy) 模型进行建模与厚度反演。

G 值大小对多光束干涉效应显著性的描述 由

$$\frac{R_{\text{eff}}(\delta)}{R_{2b}(\delta)} = \frac{|r_{01} + r_{12}e^{i\delta-\alpha}|^2}{|1 + r_{01}r_{12}e^{i\delta-\alpha}|^2} = \frac{1}{|1 + r_{01}r_{12}e^{i\delta-\alpha}|^2}.$$

α 仅是实数相于是给出

$$|1 + r_{01}r_{12}e^{i\delta-\alpha}| \in [|1 - G|, 1 + G] \Rightarrow \frac{R_{\text{eff}}}{R_{2b}} \in \left[\frac{1}{(1+G)^2}, \frac{1}{(1-G)^2}\right].$$

故最坏相对偏差 $\Delta_{\text{rel}}^{\text{max}} \approx 2G$ (当 $G \ll 1$)。故最坏相对偏差 $\Delta_{\text{rel}}^{\text{max}} \approx 2G$ (当 $G \ll 1$) 若要求两模型相差不超过约 2%、10%、20%，对应

$$G \lesssim 0.01 \ 0.05 \ 0.1$$

5.3.3 多光束干涉对计算求解精度的影响

由反射谱随机噪声引起的计算误差 在直接求解厚度 d 时，我们需要确定反射光谱的极值点位置，因此求解精度直接取决于参与计算的周期极值谷点 $\tilde{\nu}_j$ 估算精度。由于实验数据（这里指反射率曲线

或重建的样条曲线) 往往存在随机高频小幅噪声的叠加, 这造成了参与计算的极值点 $\tilde{\nu}_j$ 和真实极值谷点的偏差, 进而降低了计算精度。对于实验反射率曲线 R_0 , 我们建立如下模型:

$$R_0(\tilde{\nu}) = f(\tilde{\nu}) + \varepsilon(\tilde{\nu}),$$

其中 f 是真实的完全符合理论公式的平滑真反射谱, ε 是小幅随机噪声 (近似独立)。对真极值 $\tilde{\nu}_0$ 附近, 泰勒展开

$$f(\tilde{\nu}) \approx f(\tilde{\nu}_0) + \frac{1}{2}f''(\tilde{\nu}_0)(\tilde{\nu} - \tilde{\nu}_0)^2 \quad (\text{极值点, } f'(\tilde{\nu}_0) = 0).$$

在“小噪声 + 局部二次”近似下, 极值位置估计的随机误差一阶主导为

$$\tilde{\nu}_0 \approx \tilde{\nu} - \frac{\varepsilon'(\tilde{\nu}_0)}{f''(\tilde{\nu}_0)}.$$

$$\text{Var}(\tilde{\nu}) \approx \frac{\text{Var}[\varepsilon'(\tilde{\nu}_0)]}{(f''(\tilde{\nu}_0))^2}.$$

由此可见, 真实反射率曲线的二阶导数直接控制了 $\tilde{\nu}$ 取值与真值的偏差。即二阶导数越小, 随机误差传导到 ν 放大的倍数更大。

真实多光束干涉下建立的模型相比双光束干涉下建立的模型, 其求解厚度 d 时会对反射率曲线上非平滑的随机噪声更敏感。原因在于其相对频数 $\tilde{\nu}$ 的二阶导数更小。

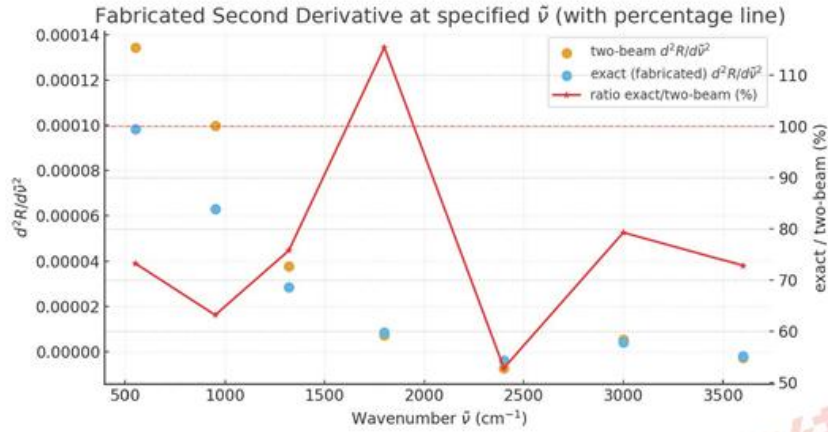


图 8: 在极值点上多/双光束干涉模型给出的二阶导, 及其比值

如图所示是附件 3, 4 参与波谷直接法计算的点上双光束与多光束模型反射率的二阶导数, 及其比值, 表中是其数值对比情况。由图中情况可以看出, 在大多数极值点上, 多光束模型的二阶导数均小于双光束模型, 根据前面的分析, 多光束干涉模型将因此对随机噪声更敏感, 进而降低计算求解精度。

5.3.4 附件 3、4 厚度计算与分析多光束干涉效应显著情况

本节通过检查附件 3、4 数据,在利用双光束干涉模型的周期波谷直接计算法和牛顿迭代法求解厚度值 d 时,所使用数据的多光束干涉效应显著性,来评估基于双光束干涉对附件 3、4 数据的两个求解是否与真实情况存在较大偏差,即附件 3、4 硅晶圆片测得数据是否出现明显的多光束干涉

[1] 利用双光束模型的附件 3、4 进行厚度计算

厚度计算: 周期波谷直接计算法

由多光束干涉模型的反射率表达式知,反射率同样随频数存在周期波动,因而周期波谷直接计算法同样适用于多光束干涉。这里先以问题 2 中周期波谷直接计算法的流程对附件 3、4 数据进行计算,再分析多光束干涉效应的显著性

用滑动窗口样条拟合硅晶片的反射率随波数的变化便于取出极值点,对所有取出的波谷按照顺序,用相邻两波谷频数来计算外延层厚度,得到多个外延层厚度,同样利用 3σ 原则剔除差距较大可疑值之后对其进行统计分析。

表 4: 计算得到的厚度 d (标准偏差大于 3σ 的异常值标记黑体)

厚度 d 值 (μm)	
入射角 10°	入射角 15°
4.090459	4.028401
3.615786	3.569042
3.478762	3.465043
3.488981	3.280644
3.372371	3.559969
3.406687	3.370166
3.317398	3.433889

最终用直接计算法的厚度将是通过统计方法处理以上数据的结果:



表 5: 剔除异常值之后硅晶片外延层厚度值的统计值

统计量	入射角 10°	入射角 15°
数据点数 L	6	5
自由度 n	5	4
平均厚度	3.446664	3.457267
标准偏差 σ_d	0.105120	0.079039
标准误差 $\sigma_{\bar{d}}$	0.042915	0.039519
$t_{\text{crit}}(95\%)$	2.570582	3.182446
CI _{95%} (low)	3.336348	3.331499
CI _{95%} (high)	3.556980	3.583035
离散系数 CV(%)	3.049892	2.286157

在 15° 入射角 CV 小于 3%，10° 入射角 CV 略大于 3% 而小于 5%，可以发现厚度计算的可靠性比较好。并根据表 5 的计算结果，利用两个入射角下得到的平均厚度和标准偏差，计算得最终的值：

$$\bar{d} = 3.453, \sigma = 0.063$$

厚度计算：牛顿迭代反演参数法

牛顿迭代反演求解厚度 我们首先利用问题 2 中类似的迭代方式，学习到一个收敛厚度值和相应的硅掺杂浓度。

反演得到硅晶片的厚度以及薄膜和衬底的掺杂浓度。



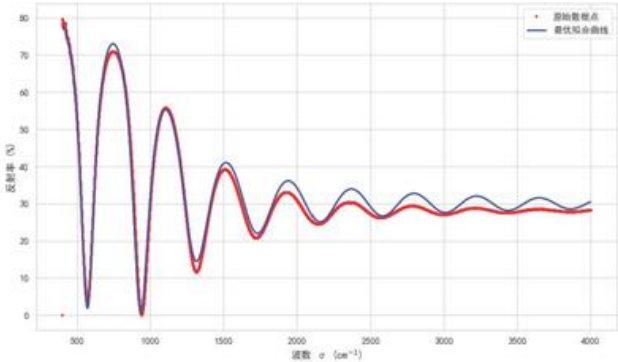


图 9: 反演参数曲线与真实硅晶片 10 度入射角反射率曲线的匹配情况

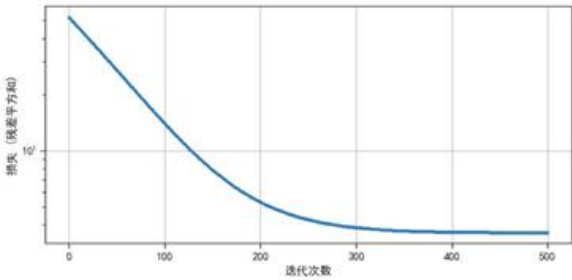


图 10: 高斯-牛顿迭代误差收敛情况

这里展示以 10 度入射时（附件 1）的牛顿迭代曲线拟合情况以及误差收敛情况，得到外延层厚度 $d = 3.414\mu\text{m}$

表 6: 反演得到的参数（硅晶片入射角 10°）

反演参数	反演值
外延层厚度 $d(\mu\text{m})$	3.414
外延层掺杂浓度 $N_1(\text{cm}^{-3})$	3.94×10^{17}
衬底掺杂浓度 $N_2(\text{cm}^{-3})$	3.80×10^{19}

[2] 根据计算结果分析多光束效应显著程度

显著程度：物理量必要条件和 G 值检验
首先考察顶界面（空气 → 外延层）反射强弱情况，这部分效应随 n_1 的模长变的显著

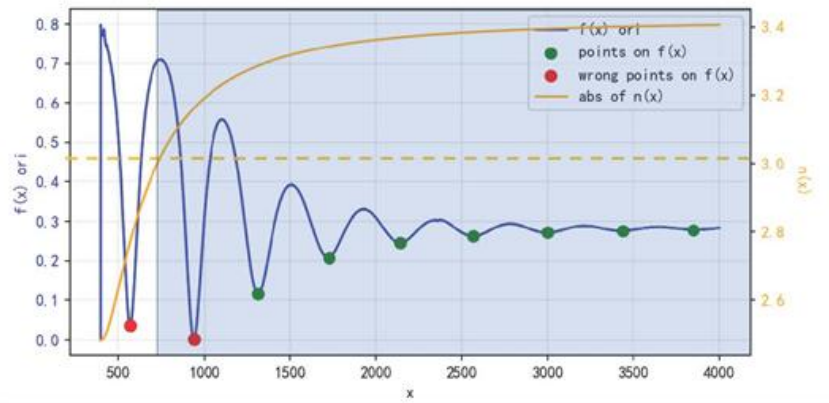


图 11: 附件 3 直接求解数据点对应的外延层折射率情况

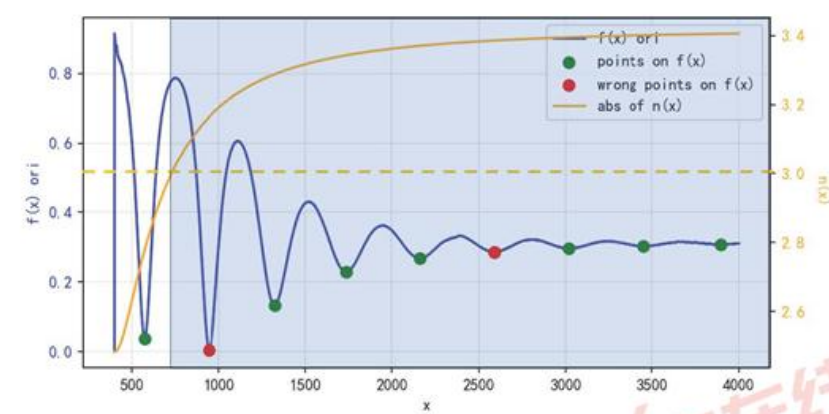


图 12: 附件 4 直接求解数据点对应的外延层折射率情况

图中用与问题 2 相同的样条插值处理法确定极值点，利用周期波谷直接计算厚度值，并通过 3σ 法则去掉了参与计算所得结果偏差过大的点（红色点，该操作合理性在“模型检验与合理性”中有严格推导和检验），可见在两次计算厚度 d 中，参与计算的多数波谷点的折射率都落在 $n > 3$ 区间内（蓝

色区域)。
接着考虑外延层中吸收强度的情况，这直接由吸收系数 $\kappa_1 = \Im(n_1)$ 表征。类似的我们展示在所用波谷数据点上吸收系数的大小情况。

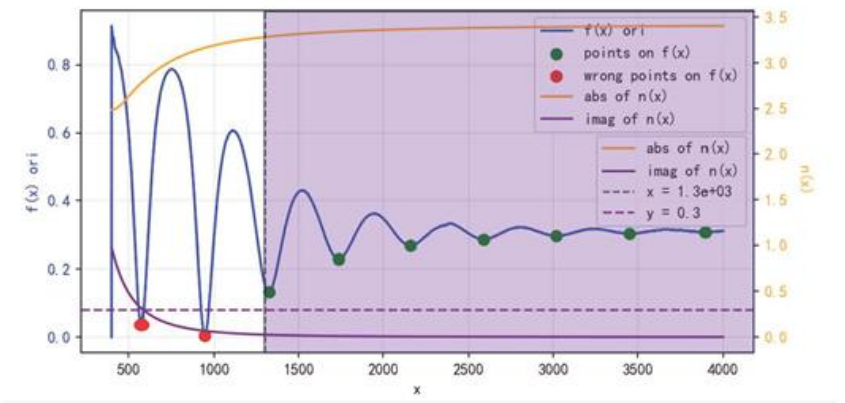


图 13: 附件 3 直接求解数据点对应的外延层吸收率情况

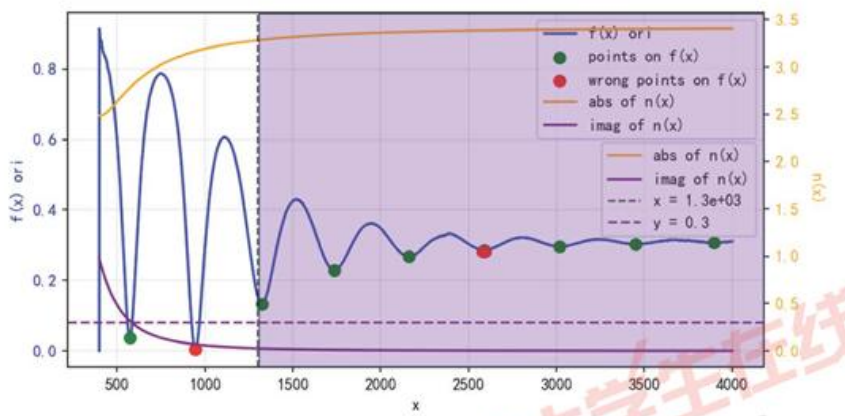


图 14: 附件 4 直接求解数据点对应的外延层吸收率情况

可以明显看出，所有取出的波谷点吸收率都小于 0.3(紫色虚线对应右侧横坐标)，参与计算的绝

大多数点 (波数大于 1300, 紫色区域) 吸收率都小于 0.05。

依照以上对两个量的估计 (我们取使得多光束干涉效应最小的估计, $n_1 = 3, \kappa_1 = 0.05$), 我们计算出附件 3, 4 对应的 G 值分别为:

0.036, 0.030 这表明多光束和双光束模型在反射率曲线上差距最多不超过 5%, 认为这个数值较大, 多光束干涉效应不可忽略

显著程度: 双光束和多光束曲线差异大小及各自反演拟合情况比较

采用双光束模型, 同样得到一组拟合的参数, 和所对应的均方误差。

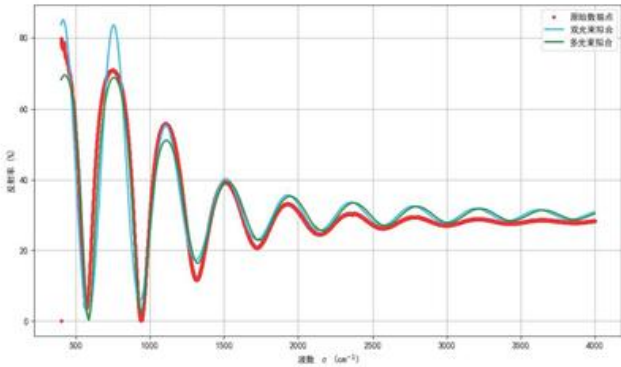


图 15: 双光束、多光束的反演拟合情况与真实硅晶片 10 度入射角反射率曲线的匹配情况

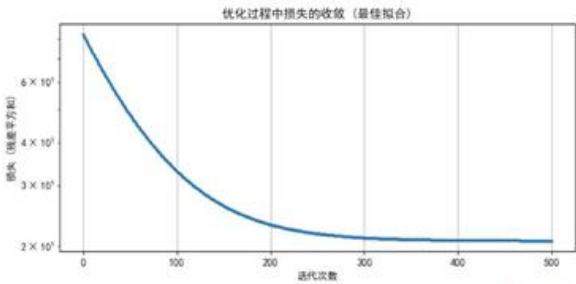


图 16: 双光束模型损失函数下高斯-牛顿迭代误差收敛情况

表 7: 两者误差对比

使用模型	均方误差 (三位有效数字)
双光束模型	20.8
多光束模型	3.62

计算得出, 收敛后双光束模型的均方误差显著大于多光束模型均方误差, 可以认为此处适合使用多光束模型进行拟合。

5.3.5 附件 1、2 分析多光束干涉效应显著情况检验

这里为简洁起见用 G 值进行判断, 代入 G 表达式计算得

$$G_1 \leq 0.016, G_2 \leq 0.010$$

, 查询 5.3.2 中对 G 值大小描述的参考中, 这意味着多光束模型和双光束模型在反射率谱上的任意一点的差距不超过 3%

我们不能简单通过反射率曲线上的最大差距来判断多光束效应对模型求解厚度数值的影响。为了进一步探究其差距对求解的影响, 下面以修正后的反射谱进行直接计算和牛顿迭代修正。

5.3.6 附件 1、2 多光束干涉影响计算精度修正

1. 周期波谷直接算法修正 多光束效应对周期波谷计算法的影响主要是影响了波谷极值点更精确值的确定 (前文已经说明), 我们考虑从附件 1, 2 的原始反射谱中还原一个更符合双光束模型的反射谱, 在其上进行的波谷极值取值和厚度计算将更可信, 误差限更小。

在前文多光束效应显著性表征量 G 的给出时, 已经证明了在入射角 θ_0 取 10° , 15° 范围内近似有

$$R_{2b}(\delta) = R_{\text{eff}}(\delta) |1 + r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}|^2 \quad (24)$$

这里我们认为附件 1、2 所展示的数据是 $R_{\text{eff}}(\delta, \bar{\nu})$ 曲线, 因此对该曲线进行修正, 得到通过上式计算得适用于双光束模型 (进而严格适用于周期波谷直接计算)。

计算结果如下:

最后用逆方差加权计算得到最终结果为:

$$d = 7.5461, \sigma = \pm 0.1801 \quad (25)$$

对比问题 2 中计算得到的 $d=7.544, \sigma=0.170$, 我们认为在附件 1, 2 中, 多光束效应是较弱的, 且可以 $d=7.544, \sigma=0.170$ 作为附件 1, 2 求解的最终结果。

2. 牛顿迭代反演法修正 使用多光束干涉给出的反射率公式拟合反射率曲线, 进而用于最小二乘损失的训练中, 以消除迭代法中模型存在的系统偏差. 结果拟合出了更贴近于原始附件 1、2 的反射率曲线, 随之输出了更可信的厚度值 d :

表 8: 统计结果

统计量	入射角 10°	入射角 15°
数据点数 L	11	15
自由度 n	10	14
平均厚度	7.575856	7.48711
标准偏差 σ_d	0.212318	0.284504
标准误差 $\sigma_{\bar{d}}$	0.067141	0.076037
$t_{\text{crit}}(95\%)$	2.262157	2.160369
$CI_{95\%}(\text{low})$	7.423973	7.322842
$CI_{95\%}(\text{high})$	7.727739	7.651377
离散系数 $CV(\%)$	2.802562	3.799922

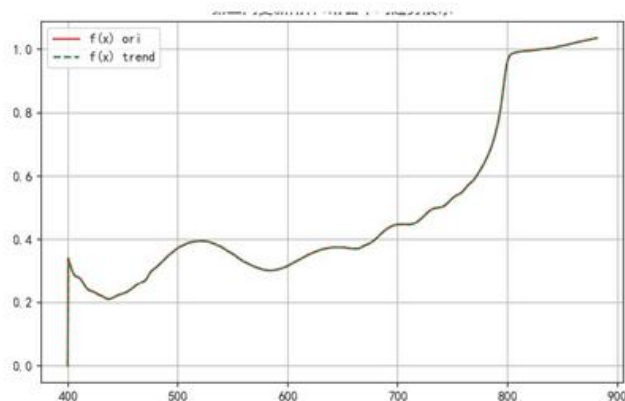


图 17: 更新牛顿迭代模型拟合到附件 1 曲线的局部

两个输出的 d 值分别为 7.560, 7.571, 与前一段计算得到的 7.546 较为吻合, 作为直接计算结果的一个较好交叉匹配。

6 模型检验及合理性分析

6.1 周期波谷直接算法去除可疑点

对于问题 2, 3, 在使用依照周期波谷直接算法时, 面对偏差较大的厚度计算值时, 我们用 3σ 法则去除了偏离均值较大的数据。这个小节里面我们证明, 所被去除的值确实存在异常, 且降低计算得到数值和真值的误差。

6.1.1 直接算法误差来源分析

由于在计算厚度 d 时, 使用的两个相邻波谷对应频数值存在较大偏差 $\Delta\tilde{\nu}$ (精准找到的极值点较难), d 的误差主要由 $\tilde{\nu}$ 数值的偏差带来, 下面分析 d 对 $\tilde{\nu}$ 的敏感程度 ($\frac{\partial d}{\partial \tilde{\nu}}$) 和误差的传递情况。

由

$$d = \frac{i}{2[\nu_j \sqrt{n^2(\nu_j) - \sin^2 \phi} - \nu_i \sqrt{n^2(\nu_i) - \sin^2 \phi}]},$$

设

$$A_k \equiv \sqrt{n^2(\nu_k) - \sin^2 \phi}, \quad n_k \equiv n(\nu_k), \quad n'_i \equiv \left. \frac{dn}{d\nu} \right|_{\nu_i}.$$

令 $F \equiv \nu_j A_j - \nu_i A_i$ (于是 $d = \frac{i}{2F}$),

对 ν_i 的灵敏度

$$\frac{\partial d}{\partial \nu_i} = -\frac{i}{2} \frac{1}{F^2} \frac{\partial F}{\partial \nu_i} = \frac{i}{2F^2} \left(A_i + \nu_i \frac{n_i n'_i}{A_i} \right).$$

等价地, 用原式分母 $D \equiv 2[\nu_j A_j - \nu_i A_i]$ 表示为

$$\frac{\partial d}{\partial \nu_i} = \frac{i}{D^2} \left(A_i + \frac{\nu_i n_i n'_i}{A_i} \right), \quad A_i = \sqrt{n_i^2 - \sin^2 \phi}, \quad n'_i = \left. \frac{dn}{d\nu} \right|_{\nu_i}.$$

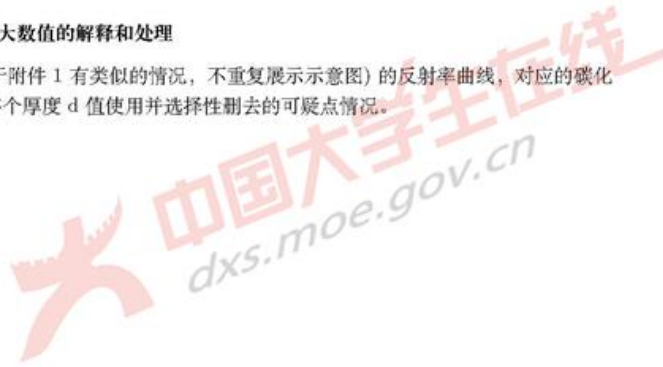
只考虑 ν_i 的不确定度 σ_{ν_i} 时, 一阶近似

$$\sigma_d \approx \left| \frac{\partial d}{\partial \nu_i} \right| \sigma_{\nu_i} = \frac{i}{D^2} \left(A_i + \frac{\nu_i n_i n'_i}{A_i} \right) \sigma_{\nu_i}$$

对敏感项 $\frac{\nu_i n_i n'_i}{A_i}$: 当 $|n'_i|$ 大 (折射率变化极快, 如靠近共振/Reststrahlen 带) 或 $A_i = \sqrt{n_i^2 - \sin^2 \phi}$ 小 (入射角较大、或 n_i 不高) 时, $\partial d / \partial \nu_i$ 将显著增大, 导致厚度 d 对 ν_i 的微小扰动极为敏感。

6.1.2 问题 2 计算中偏差大数值的解释和处理

下图展示出附件 2 (对于附件 1 有类似的情况, 不重复展示示意图) 的反射率曲线, 对应的碳化硅折射率曲线, 以及计算多个厚度 d 值使用并选择性删去的可疑点情况。



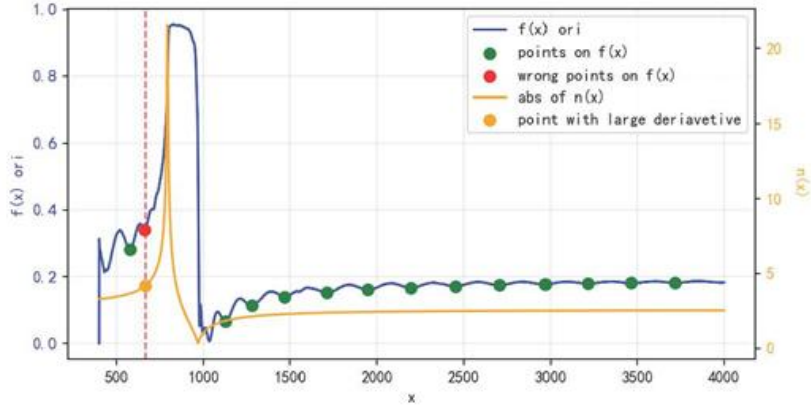


图 18: 问题 2 直接计算法可疑点去除示意图

图中可见, 偏离平均值较远并被删去的点(红点) 频数接近于碳化硅折射率共振点, 对应有较大的 $n_i, |n'_i|$ 值, 该点与前后相邻的谷底点(相邻绿点) 进行计算时 d 对频数过于敏感, 因此可能造成了计算得的厚度 d 与其他数值偏差过大的情况。

从误差传递的角度来看, 在计算中去除这一类点, 或者在其附近多次取频数值进行计算和验证, 是保证结果接近真值的合理做法。

6.1.3 问题三模型建立与求解方法总结

本题在问题一、二的基础上, 进一步考虑外延层内的多次往返反射, 建立了多光束(Airy)干涉模型。首先, 推导了等效反射振幅 $r_{\text{eff}}^{(\text{pol})}(\delta) = \frac{r_{01}^{(\text{pol})} + r_{12}^{(\text{pol})} e^{i\delta}}{1 + r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}}$ 及反射率 $R_{\text{eff}} = |r_{\text{eff}}|^2$, 并与双光束模型进行对比; 其次, 引入表征多光束效应显著性的无量纲指标 $G = |r_{01}^{(\text{pol})} r_{12}^{(\text{pol})}| \exp(-\frac{4\pi}{\lambda} \kappa_1 d \cos \theta_1)$, 给出何时必须采用多光束模型的定量判据; 再次, 从误差传播角度说明多光束模型在极值附近使二阶导更小、对噪声更敏感, 从而影响厚度求解精度; 随后在附件 3、4 上分别用周期波谷直接法与牛顿迭代法计算厚度, 并通过 G 值与均方误差对比, 验证了多光束模型在这些数据上的必要性与优越拟合效果; 最后, 对附件 1、2 给出基于 $R_{2b}(\delta) = R_{\text{eff}}(\delta) |1 + r_{01}^{(\text{pol})} r_{12}^{(\text{pol})} e^{i\delta}|^2$ 的谱线修正流程, 重新进行厚度反演, 得到与直接法一致且不确定度更合理的结果, 表明多光束效应在附件 1、2 中较弱但可量化修正, 在附件 3、4 中则必须显式建模。

7 模型评价与展望

7.1 模型优点

- **物理基础扎实**: 模型从菲涅尔公式出发, 结合 Drude 折射率模型与光学声子修正项, 较完整地刻画了材料在中远红外波段下的光学性质, 保证了物理意义的准确性。
- **方法多样互补**: 周期波谷直接计算法和牛顿迭代反演法各具优势。前者提供厚度的显式公式, 计算简洁直观; 后者通过迭代优化拟合实验曲线, 精度更高。两者结果能够相互印证, 提高了计算的可靠性。
- **多光束效应分析全面**: 引入多光束干涉的等效反射率表达式和表征参数 G , 不仅揭示了多光束效应的物理来源, 还建立了定量判断标准, 能够清晰评估其对厚度计算的影响。
- **修正方法可行**: 通过推导 $R_{2\phi}(\delta) = R_{\text{eff}}(\delta) |1 + r_{01}r_{12}e^{i\delta}|^2$ 的修正关系, 对实验反射率进行校正, 从而降低了多光束效应引起的系统偏差。
- **结果可信度高**: 计算过程中引入统计分析 (均值、标准差、置信区间、变异系数等), 确保了厚度计算不仅有理论支撑, 还有数值可靠性验证。

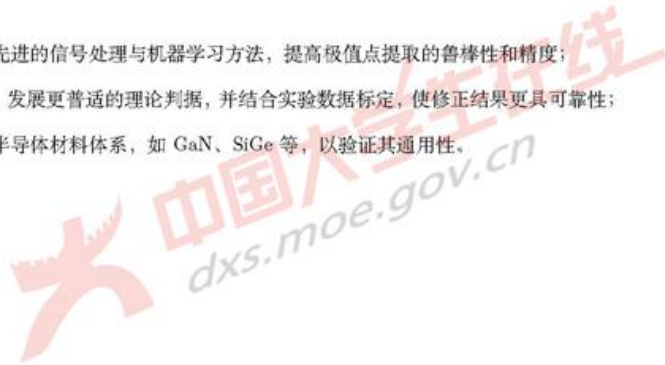
7.2 模型缺点

- **参数依赖性强**: 折射率模型所需的材料参数多取自文献, 实际样品的温度、缺陷、表面粗糙度等因素未完全纳入, 可能导致与真实情况有偏差。
- **数据处理方法有限**: 对实验曲线的平滑与极值点提取主要依赖三次样条插值, 尚未系统引入鲁棒滤波或频域降噪方法, 在高噪声条件下可能失效。
- **多光束修正仍近似**: 尽管引入 G 参数进行修正, 但该方法本质上仍是基于近似展开, 尚需更多实验数据与数值模拟来验证其适用范围。

7.3 未来展望

未来研究可以在以下方向上进一步完善:

1. 在折射率建模中引入温度效应、缺陷态以及更高精度的实验测量数据, 提升模型对实际器件样品的适用性;
2. 在数据处理上应用更先进的信号处理与机器学习方法, 提高极值点提取的鲁棒性和精度;
3. 在多光束干涉分析上, 发展更普适的理论判据, 并结合实验数据标定, 使修正结果更具可靠性;
4. 推广模型应用到更多半导体材料体系, 如 GaN、SiGe 等, 以验证其通用性。



8 物理常数与材料参量

表 9: 物理常数与材料参量表

符号	含义与数值
e	元电荷, $1.602176634 \times 10^{-19}$ C
ε_0	真空介电常数, $8.8541878128 \times 10^{-12}$ F/m
m_e	电子静止质量, $9.1093837015 \times 10^{-31}$ kg
c_0	真空光速, 2.99792458×10^8 m/s
ε_∞	碳化硅高频介电常数, 6.56
$\tilde{\nu}_T$	碳化硅 TO 光学声子波数, 794.0 cm^{-1}
$\tilde{\nu}_L$	碳化硅 LO 光学声子波数, 970.0 cm^{-1}
Γ_{ph}	声子阻尼参数, 5.5 cm^{-1}

参考文献

- [1] DeepSeek. DeepSeek-R1-671B . 深度求索 (DeepSeek), 2025. 使用日期: 2025-09-07.
- [2] ChatGPT. ChatGPT-5. OpenAI, 2025. 使用日期: 2025-09-06.
- [3] S. Basu, B. J. Lee, and Z. M. Zhang "Infrared radiative properties of heavily doped silicon at room temperature," *Journal of Heat Transfer*, vol. 132, no. 2, p. 023301, 2010. doi: 10.1115/1.4000171.
- [4] X. Tang, Y. Zhang, Z. Li, *et al.*, "Thickness determination of 4H-SiC epitaxial films by infrared reflectance," in *Proc. IEEE Int. Conf. Electron Devices and Solid-State Circuits (EDSSC)*, 2010, pp. 5394259. doi: 10.1109/EDSSC.2009.5394259.
- [5] P. J. Severin, "On the infrared thickness measurement of epitaxially grown silicon layers," *Applied Optics*, vol. 9, no. 10, pp. 2381–2386, 1970. doi: 10.1364/AO.9.002381.
- [6] S. Hava and M. Auslender, "Theoretical dependence of infrared absorption in bulk-doped silicon on carrier concentration," *Applied Optics*, vol. 32, no. 7, pp. 1122–1125, 1993. doi: 10.1364/AO.32.001122.
- [7] R. Soref, R. E. Peale, and W. Buchwald, "Longwave plasmonics on doped silicon and silicides," *Optics Express*, vol. 16, no. 9, pp. 6507–6514, 2008. doi: 10.1364/OE.16.006507.
- [8] E. Pop, S. Sinha, and K. E. Goodson, "Monte Carlo modeling of heat generation in electronic nanostructures," in *Proc. IMECE' 02, ASME Int. Mech. Eng. Congress and Exposition*, 2002, Paper No. IMECE02/HT-32124.

- [9] M. F. MacMillan, A. Henry, and E. Janzén, "Thickness determination of low doped SiC epi-films on highly doped SiC substrates," *Journal of Electronic Materials*, vol. 27, no. 4, pp. 300–306, 1998. doi: 10.1007/s11664-998-0404-9.



附录

支撑材料

- 附件1修正计算筛去异常.csv
- 附件1修正计算筛去异常_统计结果.csv
- 附件2修正计算筛去异常.csv
- 附件2修正计算筛去异常_统计结果.csv
- 物理常数和材料参量.csv
- 硅片10度拟合收敛.png
- 硅片10度折射率拟合.png
- 双光束模型拟合硅片.png
- 双光束模型拟合硅片收敛.png
- 碳化硅10度拟合.png
- 碳化硅10度拟合收敛.png
- 反射率计算.py
- 问题2碳化硅折射率物理模型.py
- 问题2统计数据计算.py
- 问题2网格-牛顿法拟合碳化硅数据.py
- 问题2样条定位极值点.py
- 问题3硅晶片折射率模型.py
- AI 工具使用详情.pdf

图 1: 支撑材料目录

代码（相似的代码未重复放入）



```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 import seaborn as sns
5 import os
6
7 from 反射率计算 import R_two_beam_sigma
8
9 # 设置Seaborn的整体风格和主题
10 sns.set_theme(style="whitegrid", palette="muted")
11 # sns.set_context("paper", font_scale=1.2, rc={"lines.linewidth": 1.5})
12
13 # 加载数据
14 data = pd.read_excel('附件1.xlsx')
15 sigma_exp = data['波数 (cm-1)'].astype(float).values
16 R_real = data['反射率 (%)'].astype(float).values / 100.0
17
18 # 物理常量 (SI units)
19 e = 1.602176634e-19 # C
20 eps0 = 8.8541878128e-12 # F/m
21 m_e = 9.1093837015e-31 # kg
22 c0 = 299_792_458.0 # m/s
23
24 # 单位转换辅助函数
25 def _cm1_to_omega(cm1):
26     """将波数 [cm-1] 转换为角频率 [rad/s]"""
27     return 2*np.pi * c0 * (cm1*100.0) # 1 cm-1 = 100 m-1
28
29 def _mu_caughey_thomas(N_cm3, mu_min, mu_max, N_c, alpha):
30     """Caughey - Thomas 迁移率模型"""
31     return mu_min + (mu_max - mu_min) / (1.0 + (N_cm3/N_c)**alpha)
32
33 def _eps_lorentz_drude_omega(omega, eps_inf, wT, wL, g_ph, v_p, g_e):
34     """Lorentz-Drude 介电函数模型:  $\epsilon(\omega) = \epsilon[1 + (wL^2 - wT^2) / (wT^2 - \omega^2 - i g_{ph} \omega)] - w_p^2 / (\omega^2 + i g_e \omega)$ """
35     omega = np.asarray(omega, dtype=complex) # 确保omega是复数
36     phonon = (wL**2 - wT**2) / (wT**2 - omega**2 - 1j*g_ph*omega)
37     drude = v_p**2 / (omega*(omega + 1j*g_e) + 0j)
38     return eps_inf*(1.0 + phonon) - drude

```



```

39
40 def n_sic_4H_signa(sigma_cm, N_cm3, m_eff_rel=0.42):
41     """
42     计算4H-SiC的复折射率 (Lorentz + Drude 模型), 波数 单位为  $\text{cm}^{-1}$ 。
43     内置固定参数和Caughey - Thomas 迁移率模型。
44     返回:  $n(\omega)$  和  $k(\omega)$ 
45     """
46     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
47
48     # 4H-SiC 固定参数
49     eps_inf = 6.56
50     sigma_L_cm = 970.1
51     sigma_T_cm = 793.9
52     Gamma_ph_cm = 5.501
53
54     # Caughey - Thomas 迁移率参数
55     mu_max = 950.0 #  $\text{cm}^2/\text{Vs}$ 
56     mu_min = 40.0 #  $\text{cm}^2/\text{Vs}$ 
57     N_c = 2e17 #  $\text{cm}^{-3}$ 
58     alpha = 0.76
59
60     # 转换为角频率
61     w = _cm1_to_omega(sigma_cm)
62     wT = _cm1_to_omega(sigma_T_cm)
63     wL = _cm1_to_omega(sigma_L_cm)
64     gph = _cm1_to_omega(Gamma_ph_cm)
65
66     # 计算Drude项的  $\omega_p$  和  $\gamma_e$ 
67     if N_cm3 > 1e-3: # N_cm3 必须大于0, 使用小阈值保证数值稳定性
68         mu = _mu_caughey_thomas(N_cm3, mu_min, mu_max, N_c, alpha) #  $\text{cm}^2/\text{Vs}$ 
69         mu_SI = mu * 1e-4 #  $\text{m}^2/\text{Vs}$ 
70         m_eff = m_eff_rel * m_e
71         w_p = np.sqrt((N_cm3*1e6) * e**2 / (eps0 * m_eff)) # rad/s
72         g_e = e / (m_eff * mu_SI) # 1/s
73     else:
74         w_p = 0.0
75         g_e = 0.0
76
77     # 计算介电函数和复折射率

```



```

78     eps = _eps_lorentz_drude_omega(w, eps_inf, vT, wL, gph, v_p, g_e)
79     n_complex = np.sqrt(eps + 0j)
80     return n_complex
81
82 # 目标函数
83 def objective_function(params_scaled, sigma_data, R_exp_data, fixed_params):
84     """
85     根据缩放参数计算残差 (R_calc - R_exp)。
86     参数:
87         params_scaled: [d_scaled, log10_N1, log10_N2]
88             d_scaled: 厚度, 单位微米
89             log10_N1: 第一层掺杂浓度 log10(N) (cm-3)
90             log10_N2: 第二层掺杂浓度 log10(N) (cm-3)
91         sigma_data: 波数数组 [cm-1]
92         R_exp_data: 实验反射率数组
93         fixed_params: 固定参数字典 (n0, i_deg, m_eff_rel)
94     返回:
95         residuals: R_calc - R_exp
96     """
97     d_scaled, log10_N1, log10_N2 = params_scaled
98
99     # 缩放参数还原为物理单位
100     d = d_scaled * 1e-6 # 微米转换为米
101     N1_cm3 = 10**log10_N1
102     N2_cm3 = 10**log10_N2
103
104     # 获取固定参数
105     n0 = fixed_params['n0']
106     i_deg = fixed_params['i_deg']
107     m_eff_rel = fixed_params['m_eff_rel']
108
109     nk1_complex = n_sic_4H_sigma(sigma_data, N_cm3=N1_cm3, m_eff_rel=m_eff_rel)
110     nk2_complex = n_sic_4H_sigma(sigma_data, N_cm3=N2_cm3, m_eff_rel=m_eff_rel)
111
112     R_calc = R_two_beam_sigma(sigma_data, n0, nk1_complex, nk2_complex, i_deg,
113                               d, pol="avg")
114
115     # 处理 R_calc 中可能出现的 NaN 或 Inf 值
116     if np.any(np.isnan(R_calc)) or np.any(np.isinf(R_calc)):

```



```

116         return np.ones_like(R_exp_data) * 1e10 # 返回一个较大的误差
117
118     return R_calc - R_exp_data
119
120 # Jacobian 矩阵计算 (数值微分)
121 def calculate_jacobian(params_scaled, func, h_rel=1e-5, **kwargs):
122     """
123     使用有限差分法计算函数 'func' 对参数 'params_scaled' 的 Jacobian 矩阵。
124     """
125     num_params = len(params_scaled)
126     initial_residuals = func(params_scaled, **kwargs)
127     num_residuals = len(initial_residuals)
128
129     J = np.zeros((num_residuals, num_params))
130
131     for i in range(num_params):
132         p_perturb = np.array(params_scaled, dtype=float)
133
134         h = h_rel * abs(p_perturb[i])
135         if h == 0: # 避免对零参数进行零步长扰动
136             h = h_rel
137
138         p_perturb[i] += h
139         perturbed_residuals = func(p_perturb, **kwargs)
140
141         J[:, i] = (perturbed_residuals - initial_residuals) / h
142     return J
143
144 # Newton-Raphson / Gauss-Newton 求解器
145 def newton_raphson_fit(initial_params_scaled, sigma_data, R_exp_data,
146                        fixed_params,
147                        max_iter=100, tol=1e-6, learning_rate=0.01, verbose=True):
148
149     params_current = np.array(initial_params_scaled, dtype=float)
150
151     history = [params_current.copy()]
152     loss_history = []

```



```

153 for iteration in range(max_iter):
154     residuals = objective_function(params_current, sigma_data, R_exp_data,
155                                   fixed_params)
156     loss = np.sum(residuals**2)
157     loss_history.append(loss)
158
159     J = calculate_jacobian(params_current, objective_function,
160                           sigma_data=sigma_data, R_exp_data=R_exp_data,
161                           fixed_params=fixed_params)
162
163     if np.any(np.isnan(J)) or np.any(np.isinf(J)) or np.any(np.isnan(
164         residuals)) or np.any(np.isinf(residuals)):
165         if verbose:
166             print(f"迭代 {iteration}: Jacobian 或残差中出现 NaN 或 Inf。停
167                 止。")
168         break
169
170     # Gauss-Newton 步长, 使用 np.linalg.lstsq 提高数值稳定性
171     try:
172         delta_params = np.linalg.lstsq(J, -residuals, rcond=None)[0]
173     except np.linalg.LinAlgError:
174         if verbose:
175             print(f"迭代 {iteration}: 遇到奇异矩阵。停止。")
176         break
177
178     params_next = params_current + learning_rate * delta_params
179
180     # 参数边界约束
181     params_next[0] = np.maximum(params_next[0], 0.01) # 最小厚度 0.01 um
182     params_next[1] = np.maximum(params_next[1], 15.0) # N1 最小 1e15 cm-3
183     params_next[2] = np.maximum(params_next[2], 15.0) # N2 最小 1e15 cm-3
184
185     # 检查收敛性
186     param_change = np.linalg.norm(params_next - params_current)
187     if param_change < tol:
188         if verbose:
189             print(f"在迭代 {iteration} 收敛。参数变化: {param_change:.2e}")
190         params_current = params_next
191         history.append(params_current.copy())

```



```

188         break
189
190     params_current = params_next
191     history.append(params_current.copy())
192
193     if verbose:
194         print(f"迭代 {iteration}, Loss: {loss:.2e}, 参数: {params_current
195               [0]:.2f}  $\mu\text{m}$ ,  $10^{\{params\_current[1]:.2f\}}$ ,  $10^{\{params\_current
196               [2]:.2f\}}$ ")
197     else:
198         if verbose:
199             print(f"在 {max_iter} 次迭代后未收敛。最终参数变化: {param_change
200                   :.2e}")
201
202     final_loss = np.sum(objective_function(params_current, sigma_data,
203                                           R_exp_data, fixed_params)**2)
204     loss_history.append(final_loss)
205     if verbose:
206         print(f"最终 Loss: {final_loss:.2e}")
207     return params_current, np.array(history), np.array(loss_history)
208
209 if __name__ == "__main__":
210     output_dir = "plot_data_for_paper"
211     os.makedirs(output_dir, exist_ok=True)
212
213     # 固定参数
214     n0 = 1.0 # 入射介质 (空气)
215     i_deg = 10.0 # 入射角 [°]
216     m_eff_rel = 0.42 # 有效质量 (相对于电子质量)
217
218     fixed_params = {
219         'n0': n0,
220         'i_deg': i_deg,
221         'm_eff_rel': m_eff_rel
222     }
223
224     # 参数搜索范围和步长
225     # d_scaled (um), log10_N1 (log10(cm^-3)), log10_N2 (log10(cm^-3))

```



```

223 d_um_start, d_um_end, d_um_steps = 5.0, 12.0, 4
224 log10_N1_start, log10_N1_end, log10_N1_steps = 16.0, 18.0, 4
225 log10_N2_start, log10_N2_end, log10_N2_steps = 17.0, 19.0, 4
226
227 d_um_grid = np.linspace(d_um_start, d_um_end, d_um_steps)
228 log10_N1_grid = np.linspace(log10_N1_start, log10_N1_end, log10_N1_steps)
229 log10_N2_grid = np.linspace(log10_N2_start, log10_N2_end, log10_N2_steps)
230
231 best_loss = np.inf
232 best_params_scaled = None
233 best_history = None
234 best_loss_history = None
235 best_initial_guess = None
236
237 total_runs = d_um_steps * log10_N1_steps * log10_N2_steps
238 run_count = 0
239
240 print(f"\n开始对 {total_runs} 组初始参数进行网格搜索...")
241 print(f"厚度 (μm) 范围: {d_um_grid}")
242 print(f"薄膜 N1 (log10) 范围: {log10_N1_grid}")
243 print(f"衬底 N2 (log10) 范围: {log10_N2_grid}")
244
245 import itertools
246 for d_init, N1_init_log10, N2_init_log10 in itertools.product(d_um_grid,
247 log10_N1_grid, log10_N2_grid):
248     run_count += 1
249     current_initial_params = [d_init, N1_init_log10, N2_init_log10]
250     print(f"\n--- 运行第 {run_count}/{total_runs} 次拟合, 初始猜测: d={
251         d_init:.2f}μm, log10(N1)={N1_init_log10:.2f}, log10(N2)={
252         N2_init_log10:.2f} ---")
253
254     # 调用拟合函数, 在网格搜索期间不打印详细迭代信息
255     final_params_scaled, history, loss_history = newton_raphson_fit(
256         current_initial_params, sigma_exp, R_real, fixed_params,
257         max_iter=200, tol=1e-6, learning_rate=0.01, verbose=False
258     )
259
260     current_loss = np.sum(objective_function(final_params_scaled, sigma_exp
261 , R_real, fixed_params)**2)

```



```

253     print(f"第 {run_count}/{total_runs} 次拟合完成。最终 Loss: {
254           current_loss:.2e}")
255
256     if current_loss < best_loss:
257         best_loss = current_loss
258         best_params_scaled = final_params_scaled
259         best_history = history
260         best_loss_history = loss_history
261         best_initial_guess = current_initial_params
262         print(f"--> 发现新的最佳拟合! Loss: {best_loss:.2e}")
263
264     print("\n=====")
265     print("          网格搜索完成          ")
266     print("=====")
267     print(f"最佳初始猜测 (d_um, log10_N1, log10_N2): {best_initial_guess}")
268     print(f"最小总 Loss: {best_loss:.2e}")
269
270
271     # 显示最佳拟合结果
272     if best_params_scaled is None:
273         print("未找到有效的拟合结果。请检查数据、模型和搜索范围。")
274         exit()
275
276     final_d_um = best_params_scaled[0]
277     final_N1_cm3 = 10**best_params_scaled[1]
278     final_N2_cm3 = 10**best_params_scaled[2]
279
280     print("\n--- 最佳拟合参数 ---")
281     print(f"厚度 d: {final_d_um:.3f} μm ({final_d_um*1e-6:.3e} m)")
282     print(f"薄膜掺杂 N1: {final_N1_cm3:.2e} cm-3")
283     print(f"衬底掺杂 N2: {final_N2_cm3:.2e} cm-3")
284
285     # 绘制反射率拟合图
286     plt.figure(figsize=(10, 6))
287
288     final_d_m = final_d_um * 1e-6
289     final_nk1 = n_sic_4H_sigma(sigma_exp, N_cm3=final_N1_cm3, m_eff_rel=
290                    m_eff_rel)
291     final_nk2 = n_sic_4H_sigma(sigma_exp, N_cm3=final_N2_cm3, m_eff_rel=
292                    m_eff_rel)

```



```

294 R_fit = R_two_bean_sigma(sigma_exp, n0, final_nk1, final_nk2, i_deg,
295                             final_d_m, pol="avg")
296 plt.plot(sigma_exp, R_real * 100, 'o', markersize=4, label='实验数据',
297          color="red")
298 plt.plot(sigma_exp, R_fit * 100, '--', linewidth=2, label='拟合结果', color=
299          "blue")
300
301 plt.xlabel('波数 ( $\text{cm}^{-1}$ )', fontsize=14)
302 plt.ylabel('反射率 (%)', fontsize=14)
303 plt.tick_params(axis='both', which='major', labelsize=12)
304 plt.legend(fontsize=12, frameon=True, shadow=True)
305 plt.grid(True)
306 plt.tight_layout()
307 plt.savefig(os.path.join(output_dir, 'reflectance_fit.png'), dpi=300)
308 plt.show()
309
310 # 保存反射率拟合数据
311 reflectance_data = pd.DataFrame({
312     'Wavenumber (cm-1)': sigma_exp,
313     'Experimental Reflectance (%)': R_real * 100,
314     'Fitted Reflectance (%)': R_fit * 100
315 })
316 reflectance_data.to_csv(os.path.join(output_dir, 'reflectance_fit_data.csv'),
317                        index=False)
318 print(f"反射率拟合数据已保存至 {os.path.join(output_dir, 'reflectance_fit_data.csv')}")
319
320 # 绘制最佳拟合的 Loss 收敛历史
321 plt.figure(figsize=(8, 5))
322 plt.plot(best_loss_history, marker='o', linestyle='--', markersize=4, color=
323          "black")
324 plt.xlabel('迭代次数', fontsize=14)
325 plt.ylabel('Loss (残差平方和)', fontsize=14)
326 plt.tick_params(axis='both', which='major', labelsize=12)
327 plt.yscale('log')
328 plt.grid(True)
329 plt.tight_layout()
330 plt.savefig(os.path.join(output_dir, 'loss_convergence.png'), dpi=300)

```



```

327 plt.show()
328
329 # 保存 Loss 历史数据
330 loss_history_df = pd.DataFrame({
331     'Iteration': np.arange(len(best_loss_history)),
332     'Loss (Sum of Squared Residuals)': best_loss_history
333 })
334 loss_history_df.to_csv(os.path.join(output_dir, 'loss_history_data.csv'),
335                        index=False)
336 print(f"Loss 历史数据已保存至 {os.path.join(output_dir, 'loss_history_data.csv')}")
337
338 # 绘制最佳拟合的参数收敛历史
339 fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
340
341 axes[0].plot(best_history[:, 0], marker='o', markersize=3, color=sns.
342              color_palette("muted")[3])
343 axes[0].set_ylabel('厚度 ( $\mu m$ )', fontsize=14)
344 axes[0].tick_params(axis='y', which='major', labelsize=12)
345 axes[0].grid(True)
346
347 axes[1].plot(best_history[:, 1], marker='o', markersize=3, color=sns.
348              color_palette("muted")[4])
349 axes[1].set_ylabel('log10(N1 cm-3)', fontsize=14)
350 axes[1].tick_params(axis='y', which='major', labelsize=12)
351 axes[1].grid(True)
352
353 axes[2].plot(best_history[:, 2], marker='o', markersize=3, color=sns.
354              color_palette("muted")[5])
355 axes[2].set_ylabel('log10(N2 cm-3)', fontsize=14)
356 axes[2].set_xlabel('迭代次数', fontsize=14)
357 axes[2].tick_params(axis='both', which='major', labelsize=12)
358 axes[2].grid(True)
359
360 plt.tight_layout()
361 plt.savefig(os.path.join(output_dir, 'parameter_convergence.png'), dpi=300)
362 plt.show()
363
364 # 保存参数收敛历史数据

```



```

361 parameter_convergence_df = pd.DataFrame({
362     'Iteration': np.arange(len(best_history)),
363     'Thickness (um)': best_history[:, 0],
364     'log10(N1)': best_history[:, 1],
365     'log10(N2)': best_history[:, 2]
366 })
367 parameter_convergence_df.to_csv(os.path.join(output_dir, '
parameter_convergence_data.csv'), index=False)
368 print(f"参数收敛数据已保存至 {os.path.join(output_dir, '
parameter_convergence_data.csv')}")

```

Listing 1: 问题 2 网格-牛顿法拟合碳化硅数据

```

369 # -*- coding: utf-8 -*-
370 """
371 该模块提供了从特定格式的CSV文件加载折射率数据，并基于该数据
372 计算单层薄膜的两光束和严格多光束反射率的功能。
373
374 主要功能：
375 - 从 refractiveindex.info 格式的 CSV 文件（包含 'w1,n' 和 'w1,k' 两部分）
376   读取折射率 n 和消光系数 k 数据。
377 - 构建一个复数折射率 n1( ) 的插值函数，其中 为波数 (cm-1)。
378 - 计算单层薄膜的两光束反射率和严格多光束反射率。
379 """
380 import re
381 import numpy as np
382 from pathlib import Path
383
384 from matplotlib import pyplot as plt
385
386
387 # =====
388 # 1) 从 CSV 读取数据并生成 n1( ) 插值器
389 # =====
390
391 def load_refractive_index_csv_two_blocks(path: str):
392     """
393     解析 refractiveindex.info 导出的 CSV 文件。
394     文件应包含两段两列表数据：
395     - 第一段表头 'w1,n' ：后续为 wavelength[um], n

```



```

396     - 第二段表头 'w1,k' : 后续为 wavelength[μm], k
397     返回: (w1_n_data, n_values), (w1_k_data, k_values)
398     其中 w1_n_data, n_values, w1_k_data, k_values 均为 numpy 数组,
399     并按波长升序排列。
400
401     若文件中缺少 'w1,k' 段, 本函数将抛出错误。
402     """
403     p = Path(path)
404     if not p.exists():
405         raise FileNotFoundError(f"文件未找到: {path}")
406
407     # 处理 BOM, 并标准化大小写/空白
408     raw = p.read_text(encoding="utf-8", errors="ignore").rstrip("\uffff")
409     lines = raw.splitlines()
410
411     # 定位表头
412     idx_n = idx_k = None
413     for i, ln in enumerate(lines):
414         s = ln.strip().rstrip("\uffff")
415         if re.match(r"~w1\s*[\s]*\s*\s*\s*$", s, flags=re.IGNORECASE):
416             idx_n = i
417         if re.match(r"~w1\s*[\s]*\s*\s*\s*$", s, flags=re.IGNORECASE):
418             idx_k = i
419
420     if idx_n is None:
421         raise ValueError("未找到 'w1,n' 表头。请检查 CSV 文件格式。")
422     if idx_k is None:
423         raise ValueError("未找到 'w1,k' 表头。对于无 k 值的场景, 请使用 `
424         build_n1_from_csv` 函数。")
425
426     def _collect_data(start_idx, end_idx):
427         wavelengths, values = [], []
428         for ln in lines[start_idx + 1:end_idx]:
429             s = ln.strip()
430             if (not s) or s.startswith("#"):
431                 continue
432             parts = [t for t in re.split(r"[\s;]+", s) if t]
433             if len(parts) < 2:
434                 continue

```



```

434         try:
435             w1_v = float(parts[0])
436             v = float(parts[1])
437         except ValueError:
438             continue
439         wavelengths.append(w1_v)
440         values.append(v)
441     wavelengths = np.asarray(wavelengths, float)
442     values = np.asarray(values, float)
443     order = np.argsort(wavelengths)
444     return wavelengths[order], values[order]
445
446     w1_n, n_vals = _collect_data(idx_n, idx_k)      # n 数据段
447     w1_k, k_vals = _collect_data(idx_k, len(lines)) # k 数据段
448
449     if w1_n.size == 0 or w1_k.size == 0:
450         raise ValueError("解析的 n 或 k 数据块为空; 请检查文件格式。")
451     return (w1_n, n_vals), (w1_k, k_vals)
452
453
454 def create_ni_interpolator_from_two_block_csv(csv_path: str, fill_mode: str = "
455     edge"):
456     """
457     基于 (w1,n) 与 (w1,k) 两段数据, 构造复数折射率  $ni() = n() + i*k()$  的插值
458     器。
459     - 输入/输出波数 单位为  $cm^{-1}$ 。
460     - 'fill_mode='edge': 对于超出数据范围的波数, 插值结果钳制到边界值。
461     - 'fill_mode='nan': 对于超出数据范围的波数, 插值结果返回 NaN。
462     若 CSV 中不存在 'w1,k' 段, 则自动以  $k=0$  处理 (适用于只有 n 值的 CSV)。
463
464     返回: (ni_of_sigma_func, (sigma_min_range, sigma_max_range))
465     其中 ni_of_sigma_func 是一个可调对象, 接受波数 ( $cm^{-1}$ ) 并返回 ni。
466     sigma_min_range 和 sigma_max_range 是有效插值范围的波数边界。
467     """
468     p = Path(csv_path)
469     raw = p.read_text(encoding="utf-8", errors="ignore").rstrip("\uffff")
470     lines = raw.splitlines()
471     norm_lines = [ln.strip().rstrip("\uffff").lower() for ln in lines]

```



```

471     idx_n = idx_k = None
472     for i, h in enumerate(norm_lines):
473         if re.match(r"~w1\s*[\s]\s*n\s*$", h):
474             idx_n = i
475         if re.match(r"~w1\s*[\s]\s*k\s*$", h):
476             idx_k = i
477
478     if idx_n is None:
479         raise ValueError("未找到 'w1,n' 表头; 请检查 CSV 文件格式。")
480
481     def _collect_data(start_idx, end_idx):
482         wavelengths, values = [], []
483         for ln in lines[start_idx + 1:end_idx]:
484             s = ln.strip()
485             if (not s) or s.startswith("#"):
486                 continue
487             parts = [t for t in re.split(r"[\s:]*", s) if t]
488             if len(parts) < 2:
489                 continue
490             try:
491                 wavelengths.append(float(parts[0]))
492                 values.append(float(parts[1]))
493             except ValueError:
494                 continue
495             wavelengths = np.asarray(wavelengths, float)
496             values = np.asarray(values, float)
497             order = np.argsort(wavelengths)
498             return wavelengths[order], values[order]
499
500     end_n_block = idx_k if idx_k is not None else len(lines)
501     w1_n, n_vals = _collect_data(idx_n, end_n_block)
502
503     if idx_k is not None:
504         w1_k, k_vals = _collect_data(idx_k, len(lines))
505         sigma_k = 1e4 / w1_k # 波长(μm) 转 波数(cm-1)
506         order = np.argsort(sigma_k)
507         sigma_k, k_vals = sigma_k[order], k_vals[order]
508     else:
509         # 如果没有 k 段, 则以 k=0 近似

```



```

510     w1_k, k_vals = w1_n.copy(), np.zeros_like(n_vals)
511     sigma_k = 1e4 / w1_k
512     order = np.argsort(sigma_k)
513     sigma_k, k_vals = sigma_k[order], k_vals[order]
514
515     sigma_n = 1e4 / w1_n
516     order = np.argsort(sigma_n)
517     sigma_n, n_vals = sigma_n[order], n_vals[order]
518
519     # 确定有效的波数取值范围 (n 和 k 数据重叠的部分)
520     valid_sigma_min = max(float(sigma_n[0]), float(sigma_k[0]))
521     valid_sigma_max = min(float(sigma_n[-1]), float(sigma_k[-1]))
522
523     def n1_of_sigma(sigma_cm_input):
524         s = np.atleast_1d(sigma_cm_input).astype(float)
525         if fill_mode == "edge":
526             n_interpolated = np.interp(s, sigma_n, n_vals, left=n_vals[0],
527                                       right=n_vals[-1])
528             k_interpolated = np.interp(s, sigma_k, k_vals, left=k_vals[0],
529                                       right=k_vals[-1])
530         else: # fill_mode == "nan"
531             n_interpolated = np.interp(s, sigma_n, n_vals)
532             k_interpolated = np.interp(s, sigma_k, k_vals)
533             # 对于超出有效范围的数据点, 设为 NaN
534             out_of_range = (s < valid_sigma_min) | (s > valid_sigma_max)
535             n_interpolated[out_of_range] = np.nan
536             k_interpolated[out_of_range] = np.nan
537             return n_interpolated + 1j * k_interpolated
538
539     return n1_of_sigma, (valid_sigma_min, valid_sigma_max)
540
541 # =====
542 # 2) 光学计算核心函数
543 # =====
544
545 def _calculate_cos_theta(n_incident, n_layer, theta_incident):
546     """
547     计算层内介质的余弦角 cos(theta_layer)。

```



```

547     遵循 Snell 定律  $n_{\text{incident}} \cdot \sin(\theta_{\text{incident}}) = n_{\text{layer}} \cdot \sin(\theta_{\text{layer}})$ 。
548     选择实部为正的平方根以确保物理有效性。
549     """
550     sin_incident = np.sin(theta_incident)
551     # 使用复数类型确保即使  $(n_{\text{inc}}/n_{\text{layer}})^2 + s^2 > 1$  也能正确处理
552     cos_layer_squared = (1 - (n_incident / n_layer)**2 * sin_incident**2).
553         astype(complex)
554     cos_layer = np.sqrt(cos_layer_squared)
555     # 物理约定: 选取实部为正的支路
556     return np.where(np.real(cos_layer) < 0, -cos_layer, cos_layer)
557
558 def _fresnel_s_polarization(n_i, n_j, cos_i, cos_j):
559     """
560     计算 s-偏振 (TE模式) 的菲涅尔反射系数 r 和透射系数 t。
561     n_i, n_j 分别为入射介质和透射介质的折射率。
562     cos_i, cos_j 分别为入射角和透射角的余弦。
563     """
564     r_s = (n_i * cos_i - n_j * cos_j) / (n_i * cos_i + n_j * cos_j)
565     t_s = 2 * n_i * cos_i / (n_i * cos_i + n_j * cos_j)
566     return r_s, t_s
567
568 def _fresnel_p_polarization(n_i, n_j, cos_i, cos_j):
569     """
570     计算 p-偏振 (TM模式) 的菲涅尔反射系数 r 和透射系数 t。
571     n_i, n_j 分别为入射介质和透射介质的折射率。
572     cos_i, cos_j 分别为入射角和透射角的余弦。
573     """
574     r_p = (n_j * cos_i - n_i * cos_j) / (n_j * cos_i + n_i * cos_j)
575     t_p = 2 * n_i * cos_i / (n_j * cos_i + n_i * cos_j)
576     return r_p, t_p
577
578 def _normalize_refractive_index_input(value, sigma_cm):
579     """
580     将折射率输入转换为与波数 `sigma_cm` 形状一致的复数数组。
581     `value` 可以是:
    
```



```

584     - 标量 (常数折射率)
585     - 可调对象: 接受 `sigma_cm` 并返回数组形式的复数折射率
586     - 数组或类似数组的对象: 与 `sigma_cm` 同形状 (或可广播)
587     """
588     s = np.atleast_1d(sigma_cm)
589     if callable(value):
590         arr = np.asarray(value(s), dtype=complex)
591     else:
592         arr = np.asarray(value, dtype=complex)
593
594     if arr.ndim == 0 or arr.size == 1: # 标量
595         return np.full(s.shape, complex(arr), dtype=complex)
596
597     # 尝试广播到 s 的形状
598     try:
599         return np.broadcast_to(arr, s.shape).astype(complex, copy=False)
600     except ValueError:
601         raise ValueError(
602             f"折射率参数形状 {arr.shape} 不能广播到波数形状 {s.shape}。"
603             "请提供标量、接受波数参数的可调函数，或与波数数组形状一致的数组。"
604         )
605
606 # =====
607 # 3) 反射率计算函数 (波数轴)
608 # =====
609
610 def calculate_R_two_beam(sigma_cm, n0, n1_ref_index, n2, incidence_angle_deg,
611 layer_thickness, polarization="avg"):
612     """
613     计算单层薄膜的两光束反射率 R( )。
614     - : 波数, 单位 cm-1。
615     - n0: 入射介质的折射率。
616     - n1_ref_index: 薄膜的复数折射率, 可以是标量、可调函数 (sigma -> n1) 或数组。
617     - n2: 基底介质的折射率。
618     - incidence_angle_deg: 入射角, 单位度。
619     - layer_thickness: 薄膜厚度, 单位米 (m)。
620     - polarization: 偏振模式, 可选 "s", "p", "avg" (默认平均)。

```



```

620
621 薄膜中的相位定义:  $\delta = 2 * n1 * d * \cos(\theta_1) * \frac{\lambda}{\lambda_0}$ , 其中  $\frac{\lambda}{\lambda_0} = 100 * \frac{cm}{nm}$  (nm
    ^-1)。
622
623 ***
624 sigma_cm = np.atleast_1d(sigma_cm).astype(float)
625 theta0 = np.deg2rad(incidence_angle_deg)
626 cos0 = np.cos(theta0)
627
628 n0s = _normalize_refractive_index_input(n0, sigma_cm)
629 n1s = _normalize_refractive_index_input(n1_ref_index, sigma_cm)
630 n2s = _normalize_refractive_index_input(n2, sigma_cm)
631
632 cos1 = _calculate_cos_theta(n0s, n1s, theta0)
633 cos2 = _calculate_cos_theta(n0s, n2s, theta0)
634
635 # 将波数从 cm^-1 转换为 m^-1
636 sigma_m = 100.0 * sigma_cm
637 # 薄膜中的相位因子
638 beta = 2 * np.pi * n1s * layer_thickness * cos1 * sigma_m
639
640 def _calculate_branch(kind):
641     if kind == "s":
642         r01, t01 = _fresnel_s_polarization(n0s, n1s, cos0, cos1)
643         r12, _ = _fresnel_s_polarization(n1s, n2s, cos1, cos2)
644         _, t10 = _fresnel_s_polarization(n1s, n0s, cos1, cos0) # 注意 n1->
            n0 接口的透射系数
645     else: # kind == "p"
646         r01, t01 = _fresnel_p_polarization(n0s, n1s, cos0, cos1)
647         r12, _ = _fresnel_p_polarization(n1s, n2s, cos1, cos2)
648         _, t10 = _fresnel_p_polarization(n1s, n0s, cos1, cos0) # 注意 n1->
            n0 接口的透射系数
649
650 # 两光束近似的反射振幅叠加
651 reflection_amplitude = r01 + t01 * t10 * r12 * np.exp(1j * 2 * beta)
652 return np.abs(reflection_amplitude)**2
653
654 if polarization.lower() == "s":
655     return _calculate_branch("s")
656 if polarization.lower() == "p":

```



```

656         return _calculate_branch("p")
657     return 0.5 * (_calculate_branch("s") + _calculate_branch("p"))
658
659
660 def calculate_R_exact(sigma_cm, n0, n1_ref_index, n2, incidence_angle_deg,
661                       layer_thickness, polarization="avg", return_complex_r=False):
662     """
663     计算单层薄膜的严格多光束反射率 R( )。
664     参数定义与 'calculate_R_two_beam' 相同。
665
666     返回：
667     - 若 'polarization='avg'': 返回平均反射率 R。
668       若 'return_complex_r=True', 则附带 (r_s, r_p) 菲涅尔复振幅反射系数。
669     - 若 'polarization='s' 或 'p': 返回对应偏振的 R。
670       若 'return_complex_r=True', 则附带对应偏振的复振幅反射系数。
671     """
672     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
673     theta0 = np.deg2rad(incidence_angle_deg)
674     cos0 = np.cos(theta0)
675
676     n0s = _normalize_refractive_index_input(n0, sigma_cm)
677     n1s = _normalize_refractive_index_input(n1_ref_index, sigma_cm)
678     n2s = _normalize_refractive_index_input(n2, sigma_cm)
679
680     cos1 = _calculate_cos_theta(n0s, n1s, theta0)
681     cos2 = _calculate_cos_theta(n0s, n2s, theta0)
682
683     sigma_m = 100.0 * sigma_cm # cm-1 to m-1
684     beta = 2 * np.pi * n1s * layer_thickness * cos1 * sigma_m
685     e_phase = np.exp(1j * 2 * beta) # 往返薄膜的相位因子
686
687     def _calculate_exact_reflection(kind):
688         if kind == "s":
689             r01, _ = _fresnel_s_polarization(n0s, n1s, cos0, cos1)
690             r12, _ = _fresnel_s_polarization(n1s, n2s, cos1, cos2)
691         else: # kind == "p"
692             r01, _ = _fresnel_p_polarization(n0s, n1s, cos0, cos1)
693             r12, _ = _fresnel_p_polarization(n1s, n2s, cos1, cos2)

```



```

694     # 严格多光束反射系数公式
695     r_complex = (r01 + r12 * e_phase) / (1 + r01 * r12 * e_phase)
696     return r_complex, np.abs(r_complex)**2
697
698     if polarization.lower() == "s":
699         r_s, R_s = _calculate_exact_reflection("s")
700         return (R_s, r_s) if return_complex_r else R_s
701     if polarization.lower() == "p":
702         r_p, R_p = _calculate_exact_reflection("p")
703         return (R_p, r_p) if return_complex_r else R_p
704
705     # 默认计算平均偏振
706     r_s, R_s = _calculate_exact_reflection("s")
707     r_p, R_p = _calculate_exact_reflection("p")
708     R_avg = 0.5 * (R_s + R_p)
709     if return_complex_r:
710         return R_avg, (r_s, r_p)
711     return R_avg
712
713
714 # =====
715 # 4) 便捷封装: 直接获取 n1() 插值器
716 # =====
717
718 def build_n1_interpolator(csv_path: str, fill_mode: str = "edge"):
719     """
720     读取 CSV 文件 (两段两列; 若无 k 段则自动 k=0),
721     返回 n1(sigma_cm) 可调插值器及其有效波数范围。
722
723     用法示例:
724         n1_func, (sig_min, sig_max) = build_n1_interpolator("material.csv")
725         reflectance = calculate_R_exact(sigma_array, n0=1.0,
726                                         n1_ref_index=n1_func, n2=3.5+0j,
727                                         incidence_angle_deg=15,
728                                         layer_thickness=800e-9)
729     """
730     return create_n1_interpolator_from_two_block_csv(csv_path, fill_mode=
731                                                       fill_mode)
732

```



```

732
733 if __name__ == "__main__":
734
735     # 指定 CSV 数据文件路径 (应包含 'w1,n' 和 'w1,k' 两段)
736     DATA_PATH = r"Larruquert.csv"
737
738     try:
739         # 1) 构造薄膜材料 n1( ) 的插值器
740         n1_interpolator, (sigma_range_min, sigma_range_max) =
741             build_n1_interpolator(DATA_PATH, fill_mode="edge")
742
743         # 2) 定义计算参数
744         sigma_values = np.linspace(400, 4000, 2200) # 波数范围 (cm-1)
745         n0_incident = 1.0 # 入射介质折射率 (例如空
746             气)
747         n2_substrate = 2.7**2 # 基底折射率 (例如 Si 的
748             近似值, 请根据实际材料替换)
749         layer_thickness_m = 2500e-9 # 薄膜厚度 (米)
750         incident_angle_deg = 20 # 入射角 (度)
751
752         # 3) 计算反射率
753         # 使用两光束近似计算反射率
754         reflectance_two_beam = calculate_R_two_beam(
755             sigma_values, n0_incident, n1_interpolator, n2_substrate,
756             incident_angle_deg, layer_thickness_m, polarization="avg"
757         )
758
759         # 使用严格多光束公式计算反射率, 并获取复数反射系数
760         reflectance_exact, (r_s_complex, r_p_complex) = calculate_R_exact(
761             sigma_values, n0_incident, n1_interpolator, n2_substrate,
762             incident_angle_deg, layer_thickness_m, polarization="avg",
763             return_complex_r=True
764         )
765
766         # 4) 绘图 (此部分被注释, 可根据需要取消注释并运行)
767         # plt.figure(figsize=(10, 6))
768         # plt.plot(sigma_values, reflectance_two_beam, label="Two-beam
769             Approximation")
770         # plt.plot(sigma_values, reflectance_exact, label="Exact Multibeam")

```



```

767     # plt.xlabel("Wavenumber (cm-1)")
768     # plt.ylabel("Reflectance R")
769     # plt.title(f"Reflectance for film '{DATA_PATH.split('/')[-1]}' on
770         substrate")
771     # plt.legend()
772     # plt.grid(True)
773     # plt.tight_layout()
774     # plt.show()
775
776     print(f"计算完成。波数范围: {sigma_values.min()} - {sigma_values.max()}
777           cm-1")
778     print(f"薄膜厚度: {layer_thickness_m*1e9} nm")
779     print(f"入射角: {incident_angle_deg}°")
780     # 打印部分结果示例
781     # print("\n部分两光束反射率结果:")
782     # print(reflectance_two_beam[:5])
783     # print("\n部分严格多光束反射率结果:")
784     # print(reflectance_exact[:5])
785
786 except ValueError as e:
787     print(f"数据解析错误: {e}")

```

Listing 2: 问题 2: 反射率数学模型计算

```

786 import re
787 from pathlib import Path
788
789 import numpy as np
790 import matplotlib.pyplot as plt
791 import pandas as pd
792
793 e = 1.602176634e-19      # C
794 eps0 = 8.8541878128e-12  # F/m
795 m_e = 9.1093837015e-31   # kg
796 c0 = 299_792_458.0       # m/s
797
798 def cm1_to_omega(cm1):
799     return 2*np.pi * c0 * (cm1*100.0)
800
801 def mobility_to_gamma_e(mu_cm2_Vs, m_eff_rel):

```



```

802     if mu_cm2_Vs is None or m_eff_rel is None:
803         return None
804     mu_SI = mu_cm2_Vs * 1e-4
805     m_eff = m_eff_rel * m_e
806     return e / (m_eff * mu_SI)
807
808 def doping_to_omega_p(N_cm3, m_eff_rel):
809     if N_cm3 is None or m_eff_rel is None:
810         return None
811     N_m3 = N_cm3 * 1e6
812     m_eff = m_eff_rel * m_e
813     return np.sqrt(N_m3 * e**2 / (eps0 * m_eff))
814
815 def eps_lorentz_drude_omega(omega, eps_inf, omega_T, omega_L, gamma_ph, omega_p
=0.0, gamma_e=0.0):
816     omega = np.asarray(omega, dtype=float)
817     phonon = (omega_L**2 - omega_T**2) / (omega_T**2 - omega**2 - 1j*gamma_ph*
omega)
818     drude = omega_p**2 / (omega * (omega + 1j*gamma_e) + 0j)
819     return eps_inf * (1.0 + phonon) - drude
820
821 def n_lorentz_drude_sigma(sigma_cm, eps_inf, sigma_T_cm, sigma_L_cm,
Gamma_ph_cm, sigma_p_cm=None, Gamma_e_cm=None, N_cm3=None, m_eff_rel=None,
mu_cm2_Vs=None):
822     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
823
824     w_T = cm1_to_omega(sigma_T_cm)
825     w_L = cm1_to_omega(sigma_L_cm)
826     g_ph = cm1_to_omega(Gamma_ph_cm)
827     w = cm1_to_omega(sigma_cm)
828
829     if sigma_p_cm is not None or Gamma_e_cm is not None:
830         w_p = cm1_to_omega(0.0 if sigma_p_cm is None else sigma_p_cm)
831         g_e = cm1_to_omega(0.0 if Gamma_e_cm is None else Gamma_e_cm)
832     else:
833         w_p = doping_to_omega_p(N_cm3, m_eff_rel) or 0.0
834         g_e = mobility_to_gamma_e(mu_cm2_Vs, m_eff_rel) or 0.0
835
836     eps = eps_lorentz_drude_omega(w, eps_inf, w_T, w_L, g_ph, w_p, g_e)

```



```

837     n_complex = np.sqrt(eps + 0j)
838     return n_complex
839
840 def n_lorentz_drude_v(v_Hz, eps_inf, sigma_T_cm, sigma_L_cm, Gamma_ph_cm,
841                      sigma_p_cm=None, Gamma_e_cm=None, N_cm3=None, m_eff_rel=None, mu_cm2_Vs=
842                      None):
843     v_Hz = np.atleast_1d(v_Hz).astype(float)
844     omega = 2*np.pi * v_Hz
845
846     w_T = cm1_to_omega(sigma_T_cm)
847     w_L = cm1_to_omega(sigma_L_cm)
848     g_ph = cm1_to_omega(Gamma_ph_cm)
849
850     if sigma_p_cm is not None or Gamma_e_cm is not None:
851         w_p = cm1_to_omega(0.0 if sigma_p_cm is None else sigma_p_cm)
852         g_e = cm1_to_omega(0.0 if Gamma_e_cm is None else Gamma_e_cm)
853     else:
854         w_p = doping_to_omega_p(N_cm3, m_eff_rel) or 0.0
855         g_e = mobility_to_gamma_e(mu_cm2_Vs, m_eff_rel) or 0.0
856
857     eps = eps_lorentz_drude_omega(omega, eps_inf, w_T, w_L, g_ph, w_p, g_e)
858     return np.sqrt(eps + 0j)
859
860 def _cm1_to_omega(cm1):
861     return 2*np.pi * c0 * (cm1*100.0)
862
863 def _eps_lorentz_drude_omega(omega, eps_inf, w_T, w_L, g_ph, w_p, g_e):
864     phonon = (w_L**2 - w_T**2) / (w_T**2 - omega**2 - 1j*g_ph*omega)
865     drude = w_p**2 / (omega*(omega + 1j*g_e) + 0j)
866     return eps_inf*(1.0 + phonon) - drude
867
868 def _mu_caughy_thomas(N_cm3, mu_min, mu_max, N_c, alpha):
869     return mu_min + (mu_max - mu_min) / (1.0 + (N_cm3/N_c)**alpha)
870
871 def n_sic_4H_sigma(sigma_cm, N_cm3=1e17, m_eff_rel=0.42, use_drude=True):
872     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
873
874     eps_inf = 6.56
875     sigma_L_cm = 970.1

```



```

913         if re.match(r"~w1\s*[,;\s]\s*k\s*$", s, flags=re.I):
914             idx_k = i
915
916     if idx_n is None:
917         raise ValueError("Header 'w1,n' not found.")
918     if idx_k is None:
919         raise ValueError("Header 'w1,k' not found.")
920
921     def collect(si, ei):
922         w1, val = [], []
923         for ln in lines[si+1:ei]:
924             s = ln.strip()
925             if (not s) or s.startswith("#"):
926                 continue
927             parts = [t for t in re.split(r"[,\s;]+", s) if t]
928             if len(parts) < 2:
929                 continue
930             try:
931                 w1_v = float(parts[0]); v = float(parts[1])
932             except ValueError:
933                 continue
934             w1.append(w1_v); val.append(v)
935         w1 = np.asarray(w1, float); val = np.asarray(val, float)
936         order = np.argsort(w1)
937         return w1[order], val[order]
938
939     w1_n, n_vals = collect(idx_n, idx_k)
940     w1_k, k_vals = collect(idx_k, len(lines))
941     if w1_n.size == 0 or w1_k.size == 0:
942         raise ValueError("Parsed empty n or k block; check file formatting.")
943     return (w1_n, n_vals), (w1_k, k_vals)
944
945 def make_ni_interpolator_from_two_block_csv(csv_path: str, fill: str = "edge"):
946     p = Path(csv_path)
947     raw = p.read_text(encoding="utf-8", errors="ignore").rstrip("\uffff")
948     lines = raw.splitlines()
949     norm = [ln.strip().rstrip("\uffff").lower() for ln in lines]
950
951     idx_n = idx_k = None

```



```

982 for i, h in enumerate(norm):
983     if re.match(r"~w1\s*[\s]\s*n\s*$", h): idx_n = i
984     if re.match(r"~w1\s*[\s]\s*k\s*$", h): idx_k = i
985 if idx_n is None:
986     raise ValueError("未找到 'w1,n' 表头; 请检查 CSV")
987
988 def collect(si, ei):
989     w1, val = [], []
990     for ln in lines[si+1:ei]:
991         s = ln.strip()
992         if (not s) or s.startswith("#"):
993             continue
994         parts = [t for t in re.split(r"[\s;]+", s) if t]
995         if len(parts) < 2:
996             continue
997         try:
998             w1.append(float(parts[0])); val.append(float(parts[1]))
999         except ValueError:
1000             continue
1001     w1 = np.asarray(w1, float); val = np.asarray(val, float)
1002     order = np.argsort(w1)
1003     return w1[order], val[order]
1004
1005 end_n = idx_k if idx_k is not None else len(lines)
1006 w1_n, n_vals = collect(idx_n, end_n)
1007
1008 if idx_k is not None:
1009     w1_k, k_vals = collect(idx_k, len(lines))
1010     sigma_k = 1e4 / w1_k
1011     order = np.argsort(sigma_k)
1012     sigma_k, k_vals = sigma_k[order], k_vals[order]
1013 else:
1014     w1_k, k_vals = w1_n.copy(), np.zeros_like(n_vals)
1015     sigma_k = 1e4 / w1_k
1016     order = np.argsort(sigma_k)
1017     sigma_k, k_vals = sigma_k[order], k_vals[order]
1018
1019 sigma_n = 1e4 / w1_n
1020 order = np.argsort(sigma_n)

```



```

991     sigma_n, n_vals = sigma_n[order], n_vals[order]
992
993     sig_min = max(float(sigma_n[0]), float(sigma_k[0]))
994     sig_max = min(float(sigma_n[-1]), float(sigma_k[-1]))
995
996     def ni_of_sigma(sigma_cm):
997         s = np.atleast_1d(sigma_cm).astype(float)
998         if fill == "edge":
999             n_i = np.interp(s, sigma_n, n_vals, left=n_vals[0], right=n_vals
1000                [-1])
1001             k_i = np.interp(s, sigma_k, k_vals, left=k_vals[0], right=k_vals
1002                [-1])
1003         else:
1004             n_i = np.interp(s, sigma_n, n_vals)
1005             k_i = np.interp(s, sigma_k, k_vals)
1006             out = (s < sig_min) | (s > sig_max)
1007             n_i[out] = np.nan; k_i[out] = np.nan
1008             return n_i + 1j*k_i
1009
1010     return ni_of_sigma, (sig_min, sig_max)
1011
1012 def _cos_theta(n_inc, n_layer, theta_inc):
1013     s = np.sin(theta_inc)
1014     ct = np.sqrt((1 - (n_inc/n_layer)**2 * s**2).astype(complex))
1015     return np.where(np.real(ct) < 0, -ct, ct)
1016
1017 def _fresnel_s(n_i, n_j, cos_i, cos_j):
1018     r = (n_i*cos_i - n_j*cos_j) / (n_i*cos_i + n_j*cos_j)
1019     t = 2*n_i*cos_i / (n_i*cos_i + n_j*cos_j)
1020     return r, t
1021
1022 def _fresnel_p(n_i, n_j, cos_i, cos_j):
1023     r = (n_j*cos_i - n_i*cos_j) / (n_j*cos_i + n_i*cos_j)
1024     t = 2*n_i*cos_i / (n_j*cos_i + n_i*cos_j)
1025     return r, t
1026
1027 def _to_n_sigma(val, sigma_cm):
1028     s = np.atleast_1d(sigma_cm)
1029     if callable(val):

```



```

1028     arr = np.asarray(val(s), dtype=complex)
1029 else:
1030     arr = np.asarray(val, dtype=complex)
1031
1032 if arr.ndim == 0 or arr.size == 1:
1033     return np.full(s.shape, complex(arr), dtype=complex)
1034
1035 try:
1036     return np.broadcast_to(arr, s.shape).astype(complex, copy=False)
1037 except ValueError:
1038     raise ValueError(
1039         f"*n 参数形状 {arr.shape} 不能广播到 sigma 形状 {s.shape}。*
1040         "请传标量、可调用 n(sigma), 或与 sigma 同形状数组。"
1041     )
1042
1043 def R_two_beam_sigma(sigma_cm, n0, ni_fun_or_const, n2, i_deg, d, pol="avg"):
1044     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
1045     theta0 = np.deg2rad(i_deg); cos0 = np.cos(theta0)
1046
1047     n0s = _to_n_sigma(n0, sigma_cm)
1048     n1s = _to_n_sigma(ni_fun_or_const, sigma_cm)
1049     n2s = _to_n_sigma(n2, sigma_cm)
1050
1051     cos1 = _cos_theta(n0s, n1s, theta0)
1052     cos2 = _cos_theta(n0s, n2s, theta0)
1053
1054     sigma_m = 100.0 * sigma_cm
1055     beta = 2*np.pi * n1s * d * cos1 * sigma_m
1056
1057     def _branch(kind):
1058         if kind == "s":
1059             r01, t01 = _fresnel_s(n0s, n1s, cos0, cos1)
1060             r12, _ = _fresnel_s(n1s, n2s, cos1, cos2)
1061             _, t10 = _fresnel_s(n1s, n0s, cos1, cos0)
1062         else:
1063             r01, t01 = _fresnel_p(n0s, n1s, cos0, cos1)
1064             r12, _ = _fresnel_p(n1s, n2s, cos1, cos2)
1065             _, t10 = _fresnel_p(n1s, n0s, cos1, cos0)
1066     E = r01 + t01*t10*r12*np.exp(1j*2*beta)

```



```

1067         return np.abs(E)**2
1068
1069     if pol == "s": return _branch("s")
1070     if pol == "p": return _branch("p")
1071     return 0.5*(_branch("s")+_branch("p"))
1072
1073 def R_exact_sigma(sigma_cm, n0, n1_fun_or_const, n2, i_deg, d, pol="avg",
1074                  return_r=False):
1075     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
1076     theta0 = np.deg2rad(i_deg); cos0 = np.cos(theta0)
1077
1078     n0s = _to_n_sigma(n0, sigma_cm)
1079     n1s = _to_n_sigma(n1_fun_or_const, sigma_cm)
1080     n2s = _to_n_sigma(n2, sigma_cm)
1081
1082     cos1 = _cos_theta(n0s, n1s, theta0)
1083     cos2 = _cos_theta(n0s, n2s, theta0)
1084
1085     sigma_m = 100.0 * sigma_cm
1086     beta = 2*np.pi * n1s * d * cos1 * sigma_m
1087     e = np.exp(1j * 2 * beta)
1088
1089     def _rexact(kind):
1090         if kind == "s":
1091             r01, _ = _fresnel_s(n0s, n1s, cos0, cos1)
1092             r12, _ = _fresnel_s(n1s, n2s, cos1, cos2)
1093         else:
1094             r01, _ = _fresnel_p(n0s, n1s, cos0, cos1)
1095             r12, _ = _fresnel_p(n1s, n2s, cos1, cos2)
1096         r = (r01 + r12 * e) / (1 + r01 * r12 * e)
1097         return r, np.abs(r)**2
1098
1099     if pol == "s":
1100         r_s, R_s = _rexact("s")
1101         return (R_s, r_s) if return_r else R_s
1102     if pol == "p":
1103         r_p, R_p = _rexact("p")
1104         return (R_p, r_p) if return_r else R_p

```



```

1105     r_s, R_s = _reexact("s")
1106     r_p, R_p = _reexact("p")
1107     R_avg = 0.5 * (R_s + R_p)
1108     if return_r:
1109         return R_avg, (r_s, r_p)
1110     return R_avg
1111
1112 def build_n1_from_csv(csv_path: str, fill: str = "edge"):
1113     return make_n1_interpolator_from_two_block_csv(csv_path, fill=fill)

```

Listing 3: 问题 2: 碳化硅的折射率和掺杂的物理模型

```

1114 import pandas as pd
1115 from matplotlib import pyplot as plt
1116 plt.rcParams['font.sans-serif'] = ['SimHei'] # for Windows
1117 plt.rcParams['axes.unicode_minus'] = False # 避免负号显示成方块
1118 import numpy as np
1119 from scipy.interpolate import UnivariateSpline
1120 from scipy.signal import find_peaks
1121 from scipy.optimize import newton
1122
1123
1124 def robust_sigma(z):
1125     med = np.median(z)
1126     return 1.4826 * np.median(np.abs(z - med))
1127
1128 def find_trend_extrema(
1129     x, y,
1130     s=None, # 平滑强度; None 自动估计
1131     grid=8000, # 评估网格密度
1132     min_distance=None, # 极值最小间隔 (x单位); None 自动设 5% 跨度
1133     min_prominence=None, # 极值最小突出度 (y单位); None 自动设 3
1134     min_curvature=0.0, # |s''(x)| 阈值
1135     edge_margin=0.01 # 丢弃边缘(2%)内的极值, 减少边界效应
1136 ):
1137     x = np.asarray(x); y = np.asarray(y)
1138     idx = np.argsort(x); x, y = x[idx], y[idx]
1139
1140     # 归一化x, 增强数值稳定性
1141     xmin, xmax = x.min(), x.max()

```



```

1142 L = xmax - xmin if xmax > xmin else 1.0
1143 xs = (x - xmin) / L
1144
1145 # 初步估计 s 和 (残差规模)
1146 # 先用轻微平滑估计残差
1147 spl0 = UnivariateSpline(xs, y, k=3, s=len(x)*np.var(y)*1e-4)
1148 res = y - spl0(xs)
1149 sigma = robust_sigma(res)
1150
1151 if s is None:
1152     s = 10 * len(x) * (sigma**2) # 起步值: 可按效果加大/减小
1153 if min_distance is None:
1154     min_distance = 0.05 * L # 5% 跨度
1155 if min_prominence is None:
1156     min_prominence = 3.0 * sigma # 3
1157 else:
1158     min_prominence = min_prominence * sigma
1159
1160 # 正式平滑
1161 spl = UnivariateSpline(xs, y, k=3, s=s)
1162 dspl = spl.derivative(1)
1163 d2spl = spl.derivative(2)
1164
1165 # 在稠密网格上做峰谷筛选 (基于 prominence / distance)
1166 uu = np.linspace(0.0, 1.0, grid)
1167 yg = spl(uu)
1168
1169 # 距离阈值转为“点数距离”
1170 min_dist_pts = max(1, int((min_distance / L) * grid))
1171
1172 # 极大值: 直接在 yg 上找峰
1173 pmax, prop_max = find_peaks(yg, prominence=min_prominence, distance=
    min_dist_pts)
1174 # 极小值: 等价于在 -yg 找峰
1175 pmin, prop_min = find_peaks(-yg, prominence=min_prominence, distance=
    min_dist_pts)
1176
1177 def refine_to_stationary(u0):
1178     # 用牛顿法在连续样条上精修, 使 s'(u)=0

```



```

1179         try:
1180             u_star = float(newton(func=dspl, x0=u0, fprime=d2spl, maxiter=20))
1181             if 0.0 <= u_star <= 1.0:
1182                 return u_star
1183         except Exception:
1184             pass
1185         return float(u0)
1186
1187     out = []
1188
1189     # 处理极大值
1190     for idxp in pmax:
1191         u0 = uu[idxp]
1192         u = refine_to_stationary(u0)
1193         if (u < edge_margin) or (u > 1 - edge_margin):
1194             continue
1195         curv = float(d2spl(u)) # 极大值应 < 0
1196         if abs(curv) < min_curvature or curv >= 0:
1197             continue
1198         out.append({"type": "max", "x": xmin + u*L, "y": float(spl(u)),
1199                    "prominence": float(prop_max["prominences"][np.where(pmax
1200                                ==idxp)[0][0])],
1201                    "curvature": abs(curv)})
1202
1203     # 处理极小值
1204     for idxp in pmin:
1205         u0 = uu[idxp]
1206         u = refine_to_stationary(u0)
1207         if (u < edge_margin) or (u > 1 - edge_margin):
1208             continue
1209         curv = float(d2spl(u)) # 极小值应 > 0
1210         if abs(curv) < min_curvature or curv <= 0:
1211             continue
1212         out.append({"type": "min", "x": xmin + u*L, "y": float(spl(u)),
1213                    "prominence": float(prop_min["prominences"][np.where(pmin==
1214                                idxp)[0][0])],
1215                    "curvature": abs(curv)})
1216
1217     out.sort(key=lambda d: d["x"])

```



```

1216         return out, (lambda xq: spl((np.asarray(xq)-xmin)/L))
1217
1218
1219 df = pd.read_excel(r"附件2.xlsx", usecols=[0, 1])
1220 x = df.iloc[:, 0]
1221 y1 = df.iloc[:, 1]
1222 extrema, f = find_trend_extrema(
1223     x, y1,
1224     grid=12000, # 加密网格
1225     min_distance=0.02 * (x.max() - x.min()),
1226
1227     min_prominence=0.1, # 低门槛, 召回更多
1228     min_curvature=0.0, # 不用曲率砍点
1229     edge_margin=0.01 #
1230 )#放宽边缘
1231 # 极值点
1232 print(f"极值点个数是{len(extrema)}")
1233 ex = []
1234 for e in extrema:
1235     print(e['x'], e['y'], "curv:", e['curvature'])
1236     ex.append(e['x'])
1237
1238 xq = np.array(ex)#极值点集合
1239 yq = f(xq)#极值点对应函数值
1240
1241 ax = plt.gca()
1242 # 评估趋势曲线 (用于画图)
1243 ys = f(x)
1244
1245 stacked_array=np.vstack((x,ys)).T
1246 df = pd.DataFrame(stacked_array)
1247 #df.to_csv(r"附件1滑动平均.csv", index=False, float_format="%.6f", encoding="
1248         utf-8")
1249
1250 fig, ax = plt.subplots(figsize=(8, 5))
1251 ax.plot(x, y1, linestyle="--", label="f(x) ori", color="red", linewidth = 1.5)
1252     # 原曲线
1253 ax.plot(x, ys, linestyle="--", label="f(x) trend", linewidth = 1.5, color = "
1254     green") # 可选: 趋势曲线
1255 def filtered_indices(data):

```



```

1252     """返回n倍标准差范围内的平均值"""
1253     arr = np.array(data)
1254     mean, std = np.mean(arr), np.std(arr)
1255     condition = (arr >= mean-2*std) & (arr <= mean+2*std)
1256     all_indices = np.arange(len(data))
1257     within_indices = all_indices[condition]
1258     excluded_indices = all_indices[~condition]
1259     return within_indices, excluded_indices
1260     filtwithin_indicesered,exclued_indices = filtered_indices(yq)
1261
1262     ax.scatter(xq[filtwithin_indicesered], yq[filtwithin_indicesered], s=40, zorder
1263               =3, label="points on f(x)",color = "green") # 你要标的点
1264     ax.scatter(xq[exclued_indices], yq[exclued_indices], s=40, zorder=3, label="
1265               wrong points on f(x)",color = "red" ) # 你要标的点
1266     ax.set_title("第二问附件2滑窗平均趋势展示")
1267     ax.grid(True)
1268     ax.legend()
1269     plt.show()
1270 \end{lstlisting}
1271 \begin{lstlisting}[caption=问题2：统计分析代码]{python}
1272 import numpy as np
1273 import pandas as pd
1274 from scipy.stats import t
1275
1276 def compute_stats(d, w=None, alpha=0.05):
1277     """
1278     d : 1D array-like, 厚度序列 {d1,..., d_{L-1}}
1279     w : 1D array-like, 可选权重 (如  $\Delta_i$ )。若 None 则不计算加权平均
1280     """
1281     d = np.asarray(d, dtype=float)
1282     n = d.size # n = L - 1
1283     if n < 2:
1284         raise ValueError("样本数不足 (至少需要 2 个厚度值)")
1285
1286     # 2.6.1 中心趋势
1287     d_bar = d.mean() # 算术平均
1288     d_wbar = None
1289     if w is not None:
1290         w = np.asarray(w, dtype=float)

```



```

1289         if w.size != n:
1290             raise ValueError("权重长度与厚度序列不一致")
1291         if np.any(w < 0):
1292             raise ValueError("权重必须非负")
1293         if w.sum() == 0:
1294             raise ValueError("权重之和为 0")
1295         d_wbar = np.sum(w * d) / np.sum(w) # 加权平均 (权重与 Δ 成正比)
1296
1297     # 2.6.2 不确定度
1298     # 样本标准差:  $\sigma_d = \sqrt{1/(n-1) * \sum (d_i - \bar{d})^2}$ 
1299     sigma_d = d.std(ddof=1)
1300
1301     # 标准误差:  $\sigma_{\bar{d}} = \sigma_d / \sqrt{n}$ 
1302     se = sigma_d / np.sqrt(n)
1303
1304     # 95% 置信区间:  $\bar{d} \pm t_{\{0.025, n-1\}} * \sigma_{\bar{d}}$ 
1305     df = n - 1
1306     tcrit = t.ppf(1 - alpha/2, df)
1307     ci_low = d_bar - tcrit * se
1308     ci_high = d_bar + tcrit * se
1309
1310     # 2.6.3 可靠性: 变异系数 CV =  $\sigma_d / \bar{d} * 100\%$ 
1311     cv = sigma_d / d_bar * 100.0
1312
1313     # L = n + 1
1314     return {
1315         "L": n + 1,
1316         "n(=L-1)": n,
1317         "mean": d_bar,
1318         "weighted_mean": d_wbar,
1319         "std_sigma_d": sigma_d,
1320         "std_error": se,
1321         "t_crit(95%)": tcrit,
1322         "CI95_low": ci_low,
1323         "CI95_high": ci_high,
1324         "CV_%": cv,
1325     }
1326
1327

```



```

1328 def run_from_csv(csv_path, d_col="d", w_col="delta_nu", alpha=0.05, out_path=
None):
1329     df = pd.read_csv(csv_path)
1330     w = None
1331     d = df.iloc[1:, 0].to_numpy()
1332
1333     res = compute_stats(d, w=w, alpha=alpha)
1334     out = pd.DataFrame([res])
1335
1336     if out_path is None:
1337         import os
1338         base, ext = os.path.splitext(csv_path)
1339         out_path = base + "_统计结果.csv"
1340
1341     out.to_csv(out_path, index=False, float_format="%.6f", encoding="utf-8-sig"
)
1342     print(out)
1343     print(f"结果已保存: {out_path}")
1344
1345 run_from_csv(r"附件1滑窗平均滤波去异常.csv", d_col="d", w_col="delta_nu")
1346 \end{lstlisting}
1347 \begin{lstlisting}[caption=问题3：硅晶片的折射率和掺杂的物理模型]{python}
1348 import numpy as np
1349
1350 # ----- 物理常数 (SI) -----
1351 e = 1.602176634e-19 # C
1352 eps0 = 8.8541878128e-12 # F/m
1353 c0 = 299792458.0 # m/s
1354 m_0 = 9.1093837015e-31 # kg
1355 m_e = 9.1093837015e-31 # kg
1356 # (1) Drude: eps_inf, 有效质量
1357 eps_inf = 11.7 # 高频介电常数 (论文) [ref]
1358 m_eff = 0.27 * m_0 # n型电子有效质量 (论文) [ref]
1359 # (2) n型 Masetti 迁移率模型参数 (单位注意: cm^2/Vs & cm^-3)
1360 MU1 = 68.5
1361 MU_MAX = 1414.0
1362 MU2 = 56.1
1363 CR = 9.2e16
1364 CS = 3.41e20

```



```

1365 ALPHA = 0.711
1366 BETA = 1.98
1367
1368 # (3) Kuzmich 不完全电离 (n型, T=300K => =1)
1369 A_n = 0.0824
1370 NO_n = 1.6e18
1371 B_low = 0.4722 # N <= NO
1372 B_high = 1.23 - 0.3162 # N > NO => 0.9138
1373 import numpy as np
1374
1375
1376 # ---- 单位换算: cm^-1 -> 角频率 (rad/s) ----
1377 def _cm1_to_omega(sigma_cm):
1378     sigma_m = np.asarray(sigma_cm, dtype=float) * 100.0 # cm^-1 -> m^-1
1379     return 2.0 * np.pi * c0 * sigma_m # = 2 c
1380
1381 # ---- n型硅: Kuzmich 不完全电离 (T 300K) ----
1382 # eta = 1 - A * exp(-(B * ln(N/NO))^2)
1383 # n型: A=0.0824, NO=1.6e18 cm^-3, B = 0.4722 (N<=NO) else 0.9138
1384 def _eta_kuzmich_n(N_cm3):
1385     N = np.maximum(np.asarray(N_cm3, dtype=float), 1.0)
1386     A, NO = 0.0824, 1.6e18
1387     B = np.where(N <= NO, 0.4722, 0.9138)
1388     eta = 1.0 - A * np.exp(-(B * np.log(N / NO))**2)
1389     return np.clip(eta, 0.0, 1.0)
1390
1391 # ---- n型硅: Masetti 迁移率 (Ne) [cm^2/Vs] ----
1392 # = 1 + (max-1)/(1+(Ne/CR)^2) - 2/(1+(CS/Ne)^2)
1393 def _mu_masetti_n(Ne_cm3):
1394     MU1, MU_MAX, MU2 = 68.5, 1414.0, 56.1
1395     CR, CS = 9.2e16, 3.41e20
1396     ALPHA, BETA = 0.711, 1.98
1397     Ne = np.asarray(Ne_cm3, dtype=float)
1398     return MU1 + (MU_MAX - MU1)/(1.0 + (Ne/CR)**ALPHA) - MU2/(1.0 + (CS/Ne)**
1399         BETA)
1400
1401 # ---- Drude 介电函数 (只含自由载流子项) ----
1402 def _eps_drude_omega(omega, eps_inf, Ne_m3, m_eff, mu_m2Vs):
1403     # _p^2 = Ne e^2 / (eps0 m*)

```



```

1403     wp2 = Ne_m3 * e**2 / (eps0 * m_eff)
1404     # = e / (m* )
1405     gamma = e / (m_eff * mu_m2Vs) if np.all(mu_m2Vs > 0) else 0.0
1406     return eps_inf - wp2 / (omega * (omega + 1j*gamma))
1407
1408 # ---- 主函数: n( ) with (cm^-1) ----
1409 def n_si_sigma(sigma_cm,
1410               N_cm3=5e18,          # 掺杂浓度 (供体), 单位 cm^-3
1411               eps_inf=11.7,        # 高频介电常数
1412               m_eff_rel=0.27,      # 电子有效质量 (相对电子质量)
1413               use_drude=True,
1414               use_ionization=True):
1415     """
1416     n型 Si: Drude 复折射率 n( )+i k( ), 输入波数 = [cm^-1]。
1417     - 自由载流子 Ne: 默认用 Kuzmicz 不完全电离: Ne = eta(N) * N
1418     - 迁移率 (Ne): Masetti 模型 (n型)
1419     - 介电函数: ( ) = _inf - _p^2 / ( ( + i ) ), = 2 c
1420     返回: 复数数组 n_complex( )
1421     """
1422     sigma_cm = np.atleast_1d(sigma_cm).astype(float)
1423     omega = _cm1_to_omega(sigma_cm)
1424     # 自由电子浓度 (考虑不完全电离)
1425     if use_ionization:
1426         eta = _eta_kuzmicz_n(N_cm3)
1427         Ne_cm3 = eta * float(N_cm3)
1428     else:
1429         Ne_cm3 = float(N_cm3)
1430     Ne_m3 = Ne_cm3 * 1e6 # cm^-3 -> m^-3
1431     # 迁移率 (由自由电子浓度决定)
1432     mu_cm2Vs = _mu_masetti_n(Ne_cm3) # cm^2/Vs
1433     mu_m2Vs = mu_cm2Vs * 1e-4 # m^2/Vs
1434     # Drude 介电函数
1435     if use_drude and Ne_cm3 > 0:
1436         m_eff = m_eff_rel * m_e
1437         eps = _eps_drude_omega(omega, eps_inf, Ne_m3, m_eff, mu_m2Vs)
1438     else:
1439         eps = eps_inf + 0j
1440     # 复折射率
1441     n_complex = np.sqrt(eps + 0j)

```



```
1462 | return n_complex |
```

Listing 4: 问题 2: 间隔法计算碳化硅数据



```

874 sigma_T_cm = 793.9
875 Gamma_ph_cm = 5.501
876 mu_max = 950.0
877 mu_min = 40.0
878 N_c = 2e17
879 alpha = 0.76
880
881 w = _cm1_to_omega(sigma_cm)
882 wT = _cm1_to_omega(sigma_T_cm)
883 wL = _cm1_to_omega(sigma_L_cm)
884 gph = _cm1_to_omega(Gamma_ph_cm)
885
886 if use_drude and N_cm3 > 0:
887     mu = _mu_caughy_thomas(N_cm3, mu_min, mu_max, N_c, alpha)
888     mu_SI = mu * 1e-4
889     m_eff = m_eff_rel * m_e
890     v_p = np.sqrt((N_cm3*1e6) * e**2 / (eps0 * m_eff))
891     g_e = e / (m_eff * mu_SI)
892 else:
893     v_p = 0.0
894     g_e = 0.0
895
896 eps = _eps_lorentz_drude_omega(w, eps_inf, wT, wL, gph, v_p, g_e)
897 n_complex = np.sqrt(eps + 0j)
898 return n_complex
899
900 def load_larruquert_csv_two_blocks(path: str):
901     p = Path(path)
902     if not p.exists():
903         raise FileNotFoundError(path)
904
905     raw = p.read_text(encoding="utf-8", errors="ignore").rstrip("\uffff")
906     lines = raw.splitlines()
907
908     idx_n = idx_k = None
909     for i, ln in enumerate(lines):
910         s = ln.strip().rstrip("\uffff")
911         if re.match(r"~w\s*[\s]*\s*\s*\s*\s*", s, flags=re.I):
912             idx_n = i

```

